

**Ministry of Higher Education and Scientific Research
Abdelhamid Ibn Badis University — Mostaganem**



**Faculty of Exact Sciences and Computer Science
Department of Mathematics and Computer Science**



Field: Computer Science

**GRADUATE PROJECT REPORT IN COMPUTER SCIENCE
Speciality: Computer Systems**

THEME:

Design and Implementation of Delivery Tracking System

Student's names: « Daoud Chams »

« Kohli Joulanar »

Project Supervisor: « Mr. A Benzohra »

Academic Year: 2025–2026

Dedication

To myself — for every late night, every moment of doubt, and every time I chose to keep going when it would have been easier to stop.

To my grandmother — you have always been proud of me, even before I gave you a reason to be. Your belief in me gave me strength I did not know I had.

To my father — thank you for inspiring me to enter the world of computer science. Without your influence, I would never have found this path. This work is, in many ways, a reflection of what you planted in me.

To my mother — you mean the world to me. Your love is the foundation of everything I am and everything I will ever become. No words will ever be enough to express what you mean to me.

To my siblings Wafia, Larbi, and Saliha , you have always been by my side, filling my life with endless joy and reminding me that I am never alone. Having you is one of my greatest blessings.

To the souls of my dear aunties Zoubida and Fatiha, may you rest in peace. I carry you in my heart every single day

To the soul of my bestie Djauida you may rest in peace .I carry you in my heart and mind and I keep achieving what we dreamed in a random day .

To my aunties Hayet, Fatima,Amina,Soulef and my fav cousin Zoubida Thank you for supporting me since day one

To my entire extended family each one of you holds a special place in this journey, and in my heart

To Mr Kairouani Ismail thank you for your encouragement since I know you

To my best friend Hadjer , you are the reason I stayed in computer science. On the days I wanted to give up, you reminded me of who I was and what I was capable of. This achievement carries your name too.

To Meriem, Hanane, and Nihel ,from the very first classes to the very last minutes of this project, you made every moment unforgettable. Thank you for the jokes, the memories, the long days and the longer nights we shared together. University would not have been the same without you.

To Imane , my supporter, my safe place. Every time I hit rock bottom, you were there to pull me back up. You never gave up on me, and for that I am endlessly grateful. Thank you for joining this journey and making it so much lighter.

To Yacino my childhood friend. Some people grow with you, and you are one of them. Thank you for always being there.

To all my other friends who crossed my path during these years thank you for the moments, the kindness, and the memories and a special thank for Aya ,Imane and Amel

This work is dedicated to all of you.

Daoud Chams

Dedication

I would like to express my sincere gratitude to my beloved parents. To my dear father, thank you for your constant support and encouragement during this period. To my dear mother, thank you for helping me overcome this journey with patience and care. My heartfelt thanks also go to my family, my teachers, and my closest friend benkdadra asmaa for their continuous motivation and support. Special thanks to the professors who guided us throughout this journey, especially Mrs Drissi asmaa for her valuable help and dedication. Thank you all for being part of this achievement.

Kohli joulanar

Acknowledgements

First and foremost, all praise and gratitude are due to Allah, the Almighty and the Merciful, for granting us the strength, health, patience, and guidance necessary to accomplish this work. Without His blessings, none of this would have been possible.

We would like to express our sincere and heartfelt gratitude to our academic supervisor, Mr. A. Benzohra, for his continuous support, valuable guidance, and unwavering availability throughout the entire duration of this project. His expertise, encouragement, and dedication have been a constant source of motivation and have greatly contributed to the quality of this work.

We would also like to extend our appreciation to the members of the jury for the time and attention they have devoted to reading and evaluating this work. Their interest and constructive feedback are deeply valued.

Finally, we wish to thank everyone who has supported us, directly or indirectly, throughout this journey. Your kindness and encouragement have meant more than words can express.

Table of Contents

Table of Contents	4
List of Figures	6
List of Abbreviations	8
Resume	9
General Introduction	10
Chapter 1: General Context and Problematic	11
1.1 Introduction	11
1.2 Main Features:	11
1.2.1 Parcel Registration and Validation	11
1.2.2 Driver Assignment and Route Matching	11
1.2.3 Step-by-Step Delivery Status Tracking	12
1.2.4 Real-Time GPS Tracking	12
1.2.5 Customer Self-Service Tracking	12
1.2.6 Issue and Exception Management	12
1.3 Requirements Specification	13
1.3.1 System Actors:.....	13
1.3.2 Functional Requirements.....	13
1.3.3 Non-Functional Requirements	15
1.4 Conclusion.....	16
Chapter 2: Modelling	17
2.1 Introduction	17
2.2 Modelling Method: RUP and UML	17
2.2.1 Definition	17
2.2.2 Advantages of UML	17
2.2.3 Limitations of UML	17
2.3 Modelling.....	17
2.3.1 Use Case Specification	17
2.4 Sequence Diagrams	19
2.4.1 Client Registration Sequence Diagram.....	20
2.4.2 Client Authentication Sequence Diagram.....	21
2.4.3 Agency Staff Login Sequence Diagram	22
2.4.4 Forgot Password Sequence Diagram	22

2.4.5 User Management (Admin) Sequence Diagram	24
2.4.6 Profile Management Sequence Diagram	25
2.4.7 Parcels Management & Search Sequence Diagram	26
2.4.8 Parcel Submission & Validation (Agent) Sequence Diagram	26
2.4.9 Client Track Order Sequence Diagram	27
2.4.10 Driver Authentication Sequence Diagram	29
2.4.11 Driver Updates Status Sequence Diagram	30
2.4.12 Parcel Assignment and Delivery Sequence Diagram	31
2.4.13 Payment – Online (upload proof image) Sequence Diagram	32
2.4.14 Exception Handling by Agent Sequence Diagram	33
2.4.15 Admin Consult Profile Sequence Diagram	34
2.5 Class Diagram.....	35
2.6 State Diagram	37
2.7 Entity-Relationship Diagram (ERD)	38
2.8 Conclusion.....	39
Chapter 3: System Implementation	40
3.1 Introduction:	40
3.2 Implementation Tools:	40
3.2.1 Visual Studio Code.....	40
3.2.2 Programming Languages and Frameworks.....	40
3.2.3 Database — Supabase (PostgreSQL).....	40
3.2.4 Additional Tools	40
3.2.5 Justification of Technical Choices.....	41
3.3 Application Interfaces	41
3.3.1 Public Interface — Homepage	41
3.3.2 Authentication Interface.....	44
3.3.3 Admin Dashboard	45
3.3.4 Agent Dashboard	47
3.3.5 Driver Dashboard.....	50
3.3.6 Client Dashboard (After Login)	51
3.4 Conclusion.....	52
Bibliography.....	54

List of Figures

Figure 1: Static Context Diagram
Figure 2: Use Case Diagram — Client
Figure 3: Use Case Diagram — Agent
Figure 4: Use Case Diagram — Local Delivery Man
Figure 5: Use Case Diagram — Admin
Figure 6: Client Registration Sequence Diagram
Figure 7: Client – Authentication seq
Figure 8: Auth Flow (Client & Staff Login) seq
Figure 9: Forgot Password seq
Figure 10: User Management (Admin) seq
Figure 11: Profile Management seq
Figure 12: Parcels Management & Search seq
Figure 13: Parcel Submission & Validation (Agent) seq
Figure 14: Inter-Wilaya Transfer & Arrival Notification seq
Figure 15: Client – Track Order seq
Figure 16: Driver – Authentication seq
Figure 17: Driver – Consult Profile seq
Figure 18: Driver Updates Status seq
Figure 19: Parcel Assignment and Delivery seq
Figure 20: Payment – Cash on Delivery (COD) seq
Figure 21: Payment – Online (upload proof image) seq
Figure 22: Exception Handling by Agent seq
Figure 23: Admin – Consult Profile seq
Figure 24: class diag
Figure 25: state diag
Figure 26: Entity-Relationship Diagram (ERD)
Figure 27 : Homepage Hero Section
Figure 28 : How It Works Section
Figure 29 : Parcel Tracking Page
Figure 30 : Live GPS Tracking
Figure 31 : Reception Confirmation and Rating
Figure 32: Client Login Page
Figure 33 : taff Login Page with role selection
Figure 34 : Overview
Figure 35 : Users Management
Figure 36 : Create Staff Account Modal
Figure 37 : Parcels Overview
Figure 38 : Register Parcel Form
Figure 40: Validation Modal
Figure 41 : Driver Assignment Modal
Figure 42 : Delivery Slip with QR Code
Figure 43 : Refusal Notifications
Figure 44 : Assigned Deliveries List
Figure 45 : Refusal Modal

Figure 46 : Report Delivery Issue

Figure 47 : Client Dashboard

Figure 48 : User Profile

List of Abbreviations

API	Application Programming Interface
REST	Representational State Transfer
GPS	Global Positioning System
SPA	Single Page Application
MVC	Model View Controller
DB	Database
FK	Foreign Key
OTP	One Time Password
SMTP	Simple Mail Transfer Protocol
CSV	Comma Separated Values
PDF	Portable Document Format
UI	User Interface
UX	User Experience
JWT	JSON Web Token
OSRM	Open Source Routing Machine
QR	Quick Response (code)
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure
PHP	Hypertext Preprocessor
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
JS	JavaScript
SQL	Structured Query Language
Admin	Administrator

Resume

Efficient delivery management requires precise coordination between parcels, delivery vehicles, drivers, and customers.

In many delivery services, the lack of real-time visibility and poor data organization often result in delays, lost parcels, and reduced customer satisfaction.

This project proposes the design and implementation of a delivery tracking system that ensures transparent and reliable monitoring of parcels throughout their entire lifecycle.

The proposed system manages core entities including parcels, drivers, vehicles, and customers.

Each parcel is associated with a specific customer and assigned to a driver and vehicle for delivery.

The system enables tracking of parcel status through predefined stages such as registered, assigned, in transit, and delivered.

Drivers can update delivery progress in real time, while customers can view the current status and relevant delivery information at any time.

The expected outcome is a robust and reliable delivery tracking solution that improves operational efficiency, enhances customer transparency, and provides a solid foundation for modern logistics and parcel delivery services.

General Introduction

Delivering a parcel sounds simple yet behind every successful delivery lies a carefully coordinated chain of people, decisions, and processes.

From the moment a package is registered at an agency to the second it reaches a customer's doorstep, dozens of things must go right.

For companies managing hundreds or thousands of shipments at once, keeping that chain running smoothly is anything but trivial.

At the core of this challenge is one fundamental need: knowing where every parcel is, at every moment and making sure that information reaches the right person at the right time.

For a long time, delivery companies got by on manual methods.

Packages were logged on paper, drivers called in their updates by phone, and supervisors pieced together the day's picture from scattered spreadsheets.

When volumes were small, this worked well enough. But as the sector grew, cracks started to show. Parcels were misrouted or lost.

Customers called agencies repeatedly with no real answer. Managers made decisions based on outdated or incomplete information. The cost — in time, money, and customer trust — kept climbing.

Digital tracking systems changed the equation. A well-built delivery platform gives every stakeholder what they need: managers get a live view of their entire operation, drivers receive clear and up-to-date task lists, and customers can follow their shipments on their own no phone calls required. More than a convenience, such a system becomes the operational backbone of any serious delivery business.

This project, **DeliverIT**, was built with exactly that purpose in mind. The idea grew directly out of the realities of the Algerian delivery sector a market that has expanded rapidly over recent years, yet still largely operates without reliable digital tools.

The consequences are visible: delayed shipments, lost parcels, poor coordination between agencies, and customers left in the dark. DeliverIT was designed to address these problems head-on, built around one core principle: transparent, step-by-step parcel tracking from registration to final delivery.

This report documents the full journey of the project, structured as follows:

Chapter 1: examines the context and motivation behind the project, identifies the key problems in the current delivery landscape, and formally defines the system's requirements.

Chapter 2: presents the modeling phase, using UML diagrams and the RUP methodology to describe the system's architecture and behavior before implementation.

Chapter 3: covers the implementation itself — the technologies chosen, the technical decisions made, and a walkthrough of the application's main interfaces.

Chapter 1: General Context and Problematic

1.1 Introduction

Every day, millions of parcels are shipped worldwide, requiring a well-organized process to ensure they reach their destinations. This process involves multiple steps, from parcel registration to final delivery, and depends heavily on coordination between people and systems. One of the most critical aspects of this process is tracking — knowing exactly where each parcel is at any moment and making that information easily accessible.

In the past, delivery companies relied on manual methods such as paper records, phone communication, and basic spreadsheets. While these methods worked for smaller operations, they became inefficient as delivery volumes increased. This led to common issues like lost or delayed parcels, lack of transparency for customers, and poor operational visibility for managers.

To overcome these challenges, digital tracking systems were introduced. These systems allow companies to monitor parcels in real time, improve coordination, and provide customers with instant updates on their shipments. They also help managers oversee operations more effectively and enable drivers to follow clear delivery instructions.

The DeliverIT project was developed in response to these needs, particularly within the Algerian delivery sector. Its main objective is to provide a reliable and structured parcel tracking system. This chapter presents the overall context of the project, highlights the key problems faced in the delivery sector, introduces the proposed solution, and outlines the system's main requirements.

1.2 Main Features:

1.2.1 Parcel Registration and Validation

Agency staff register new parcels through a structured form that captures the sender's full details (name, phone number, wilaya, and commune), the receiver's full details (name, phone number, wilaya, commune, and delivery address), along with the parcel description, weight, and payment method. The system supports all 1,541 communes (baladiyat) across the 69 wilayas of Algeria, allowing precise location entry at the commune level rather than just the wilaya. Upon registration, the system automatically generates a unique tracking code for the parcel. The agent then performs a validation step, reviewing all captured information before formally confirming the parcel. For online payments, the system checks whether a payment proof has been uploaded by the client before allowing validation to proceed. An optional sender email field allows a parcel receipt to be sent automatically to the sender upon registration.

1.2.2 Driver Assignment and Route Matching

DeliverIt distinguishes between two categories of drivers: intra-wilaya drivers, who are assigned to a specific wilaya and handle local deliveries within it, and inter-wilaya drivers, who are assigned a fixed two-wilaya route and transport parcels between them (for example, Mostaganem to Alger).

When an agent assigns a parcel, the system automatically detects whether the delivery is intra or inter-wilaya by comparing the origin and destination communes. It then filters and displays only the drivers whose type and assigned route match the parcel — ensuring that an intra-wilaya driver in Oran is never shown for a parcel going to Constantine, and that an inter-wilaya driver on the Mostaganem–Alger route is only shown for parcels on that specific corridor. Each assignment is recorded in the parcel status history with a timestamp.

1.2.3 Step-by-Step Delivery Status Tracking

Every parcel progresses through a clearly defined workflow with the following statuses:

- Pending: parcel registered by the agent, awaiting validation.
- Registered: parcel validated and confirmed by the agent, awaiting driver assignment.
- Assigned: a driver has been designated for the parcel.
- Accepted: the driver has accepted the delivery assignment.
- Out for Delivery: the driver has started the delivery and is en route to the recipient.
- Delivered: the parcel has been successfully handed to the recipient.
- Confirmed: the recipient has confirmed reception through the platform.
- Failed: the delivery could not be completed, with a reason recorded by the driver.
- Refused: the driver refused the assignment, with a reason recorded; the parcel returns to Registered status for reassignment.

Each status transition is recorded in the parcel status history table with a precise timestamp. When a driver reports a failed delivery, the failure reason (such as address not found, recipient not available, or parcel refused) is saved and displayed to the client on the public tracking page.

1.2.4 Real-Time GPS Tracking

When a driver starts a delivery, the system requests access to the device's GPS location via the browser's Geolocation API. Once permission is granted, the driver's coordinates are transmitted to the server every five seconds and stored against the parcel record.

On the public tracking page, clients tracking a parcel with Out for Delivery status will see an animated orange marker on an interactive OpenStreetMap map, representing the driver's live position. The marker updates automatically every five seconds without requiring a page refresh. The map also displays the delivery route between the origin and destination communes using the OSRM open-source routing engine, which calculates and renders the actual road path rather than a straight line.

When the parcel is delivered or the delivery ends, GPS sharing stops automatically.

1.2.5 Customer Self-Service Tracking

Any visitor can track a parcel on the public homepage by entering its tracking code — no account or login is required. The tracking page displays the current parcel status through a visual progress stepper, the sender and receiver information, the delivery route on an interactive map, and the live driver location when the parcel is out for delivery.

For online payment parcels, clients can upload their payment proof directly from the tracking page. Once the parcel is delivered, the client can confirm reception and submit a star rating (1 to 5) along with an optional written comment. A printable delivery slip with a QR code linking back to the tracking page is also available for download.

Registered clients can additionally log in to a personal dashboard, which lists all parcels associated with their phone number, grouped by status, and provides direct access to each parcel's tracking page.

1.2.6 Issue and Exception Management

Drivers can report delivery issues directly from their dashboard by selecting a predefined reason: address not found, recipient not available, recipient refused delivery, damaged parcel, wrong address, or other. Upon reporting, the parcel status is updated to Failed and the reason is stored and made visible to the client on the tracking page.

When a driver refuses an assigned parcel before starting the delivery, they must also select a reason. The parcel is automatically returned to Registered status so the agent can reassign it to a different driver. The agent receives an in-app notification in real time indicating which driver refused the parcel and why, allowing them to act immediately.

1.3 Requirements Specification

This section formally defines the requirements of the DeliverIt system. The requirements are organized into functional requirements , describing what the system must do , and non-functional requirements , describing the quality standards the system must meet.

1.3.1 System Actors:

DeliverIt is designed to serve four distinct categories of users, each with a specific role and set of responsibilities within the system.

The Administrator is responsible for managing all staff accounts, monitoring system-wide activity, and maintaining the overall configuration of the platform. The administrator has full access to all parcels, all users (staff and clients), and all system statistics.

The Agent is the primary operational user within the delivery agency. They register and validate incoming parcels, assign them to the appropriate drivers based on route matching, monitor parcel statuses, and respond to driver refusal notifications.

The Driver uses the system to view their assigned deliveries, accept or refuse assignments, start deliveries with GPS sharing, mark parcels as delivered, and report delivery issues with a selected reason.

The Client accesses the system to track their parcels publicly without authentication, or through a personal dashboard when logged in. Clients can upload payment proof, confirm reception, rate their delivery experience, and manage their profile.

1.3.2 Functional Requirements

1.3.2.1 Authentication and User Management

The system must allow new clients to register by providing their full name, phone number, email address, and a password. Staff accounts must be created exclusively by the administrator self-registration is not permitted for agents or drivers. Upon creating a staff account, the system must automatically send an email to the new staff member containing their login credentials.

All users must be able to authenticate using their credentials, and the system must enforce role-based access control, restricting each user to the pages and features corresponding to their assigned role.

A password reset mechanism must be available through a secure email link. Users must be able to update their profile information, including name, phone number, and password, from their respective profile sections.

1.3.2.2 Parcel Management

Agency staff must be able to register new parcels by capturing sender and receiver details with wilaya and commune selection across all 1,541 Algerian communes, parcel description, weight, payment method, and an optional sender email. The system must automatically generate a unique tracking code upon registration.

A validation step must allow the agent to review all information before confirming the parcel. The system must then allow the agent to assign the parcel to a driver, automatically filtering available drivers by type and route compatibility. Staff must be able to search and filter parcels by tracking code, status, sender, or receiver name. The system must support printing a parcel delivery slip with a QR code from the agent dashboard.

1.3.2.3 Tracking and Monitoring

Any user must be able to track a parcel by entering its tracking code on the public homepage without requiring an account. The tracking page must display the current parcel status through a visual stepper, sender and receiver information, the delivery route on an OpenStreetMap interactive map using real road paths, and the live GPS position of the driver when the parcel is out for delivery, updated every five seconds.

When a delivery has failed, the tracking page must display the failure reason reported by the driver. The admin dashboard must provide real-time statistics including total parcels, parcels in transit, delivered, confirmed, failed, and the overall delivery success rate.

1.3.2.4 Delivery Operations

The system must provide each driver with a list of their assigned parcels, displaying receiver name, phone number, destination commune, and delivery address. Drivers must be able to accept or refuse assignments. Upon refusal, they must select a reason, and the parcel must automatically return to Registered status for reassignment.

Drivers must be able to start a delivery, triggering GPS location sharing to the server every five seconds. A "Get Directions" button must open Google Maps with the delivery address pre-filled for navigation. Drivers must be able to mark parcels as delivered or report a delivery issue with a predefined reason.

1.3.2.5 Issue and Exception Handling

Drivers must be able to report delivery failures by selecting a reason from a predefined list. The failure reason must be stored and displayed on the public tracking page so the client is informed. When a driver refuses an assignment, the agent must receive a real-time in-app notification displaying the driver name, parcel tracking code, and refusal reason, so they can promptly reassign the parcel.

1.3.2.6 Customer Features

Clients must have access to a personal dashboard listing all their parcels grouped by status, with filtering and search capabilities. They must be able to click any parcel to access its full tracking page. For online payment parcels, clients must be able to upload payment proof directly from the tracking page. Upon delivery, clients must be able to confirm reception and submit a rating from one to five stars with an optional comment. Clients must be able to update their profile information including name, phone number, and password.

1.3.2.7 Agency Operations

Agency staff must have access to a consolidated parcel list with search and status filters. The system must display which drivers are available and their assigned routes to assist in assignment decisions. Staff must receive real-time notifications when a driver refuses an assigned parcel. The agent must be able to print a delivery slip with QR code for any registered parcel. For online payment parcels, the agent must be able to view the payment proof submitted by the client from within the validation modal.

1.3.2.8 Administrator Functions

The administrator must be able to create staff accounts for agents and drivers, specifying the driver type (intra-wilaya or inter-wilaya) and assigning the appropriate wilaya or route. Separate user management views must exist for staff and clients. The administrator must have access to system-wide statistics and a complete list of all parcels with status, route, payment, and rating information.

1.3.2.9 Notifications

The system must send an automatic email receipt to the sender upon parcel registration if a sender email was provided. The email must include the tracking code and all parcel details. Staff accounts must receive a credentials email upon account creation. The system must display in-app notifications to agents when a driver refuses an assigned parcel, including the reason and tracking code.

1.3.3 Non-Functional Requirements

1.3.3.1 Performance

The system must respond to standard user interactions within three seconds under normal operating conditions. The live GPS tracking feature must update the driver's position on the client tracking page within five seconds of each coordinate transmission. The map must load and render road routes using OSRM without blocking the rest of the tracking page.

1.3.3.2 Usability

The interface must be intuitive and accessible to all user types, including drivers who must be able to perform their tasks on a mobile phone browser without formal training. All forms must display field-level validation messages in real time, guiding the user to correct any errors before submission. The platform must be fully functional on modern desktop and mobile browsers including Chrome, Firefox, Safari, and Edge.

1.3.3.3 Reliability

The system must handle unexpected errors gracefully, ensuring that failures in non-critical features such as email sending, SMS dispatch, or GPS transmission never crash the application or cause data loss. GPS sharing failures must be caught silently and retried on the next interval. Email sending failures must be logged but must not prevent parcel registration from completing successfully.

1.3.3.4 Security

Access to system features must be strictly enforced based on each user's assigned role using token-based authentication via Laravel Sanctum. All user passwords must be stored using bcrypt hashing. Staff ID prefixes must be validated against the selected role at login to prevent unauthorized access. Client tracking data must not expose internal staff identifiers or operational details beyond what is necessary for tracking purposes.

1.3.3.5 Maintainability and Scalability

The codebase follows a clean separation between the React frontend (SPA) and the Laravel backend (REST API), ensuring that each layer can be developed and extended independently. The commune and wilaya data is maintained in a dedicated data file, allowing easy updates as administrative divisions change. The driver route system is designed to support any number of wilayas and routes without structural changes to the codebase.

1.3.3.6 Technology Stack

The backend of the system is developed in PHP using the Laravel framework, following RESTful API design principles with Laravel Sanctum for authentication. The frontend is built with React.js using Vite as the build tool, with plain CSS for styling. All persistent data is stored in a PostgreSQL database hosted on Supabase. Interactive maps are rendered using the Leaflet.js library with OpenStreetMap tiles and OSRM for road routing.

1.4 Conclusion

This chapter has established the full context and motivation for the DeliverIt project. We began by examining the current state of the delivery sector in Algeria a rapidly growing market constrained by the absence of modern operational tools and the lack of real-time parcel visibility for both customers and delivery companies.

In response to these challenges, we proposed DeliverIt a centralized, web-based delivery tracking and management platform designed to bring transparency, structure, and efficiency to every stage of the parcel lifecycle. The system addresses the specific operational realities of the Algerian market, including support for all 1,541 communes across 69 wilayas, a route-based driver assignment model distinguishing between intra-wilaya and inter-wilaya deliveries, and real-time GPS tracking using open-source mapping technologies.

We described the system's principal features across four user roles Administrator, Agent, Driver, and Client and formalized the functional and non-functional requirements that guide its implementation. With this foundation in place, the following chapter moves into the implementation phase, presenting the development tools, technical architecture, and user interfaces of the DeliverIt application.

Chapter 2: Modelling

2.1 Introduction

This chapter is dedicated to the UML modelling of the **DeliverIT** system. Modelling is an essential step in the software development process: it allows us to formalise the system's architecture before writing a single line of code, ensure that all requirements identified in Chapter 1 are correctly translated into a structural design, and provide a shared reference for all members of the team. We will present the static and dynamic views of the system through a series of diagrams: use case diagrams for each actor, a static context diagram, sequence diagrams for the main use cases, and a class diagram.

2.2 Modelling Method: RUP and UML

2.2.1 Definition

The Unified Modelling Language (UML) is a standardised graphical modelling language used in software engineering and object-oriented design. It provides a set of diagram types that cover the static structure of a system (class diagrams, use case diagrams) and its dynamic behaviour (sequence diagrams, activity diagrams).

We follow the Rational Unified Process (RUP) methodology to structure our development phases, using UML as the modelling language.

2.2.2 Advantages of UML

- UML has a precise semantics and a graphical syntax composed of diagrams.
- It allows the construction of multiple views (functional, static, dynamic) of the same system.
- It is a universal language understood by software professionals worldwide.
- It facilitates the translation of textual requirements into visual models.

2.2.3 Limitations of UML

UML requires learning and experience to master.

UML is not a method in itself; it is a language that must be used within a development methodology like RUP.

Some diagrams can become complex and difficult to read for large systems.

2.3 Modelling

2.3.1 Use Case Specification

2.3.1.1 Static Context Diagram

The static context diagram (Figure 1) delimits the system boundary and identifies the four external actors that interact with DeliverIT: Client, Agent, Driver, and Admin. It shows the system as a single entity and all external external interactions at a high level.

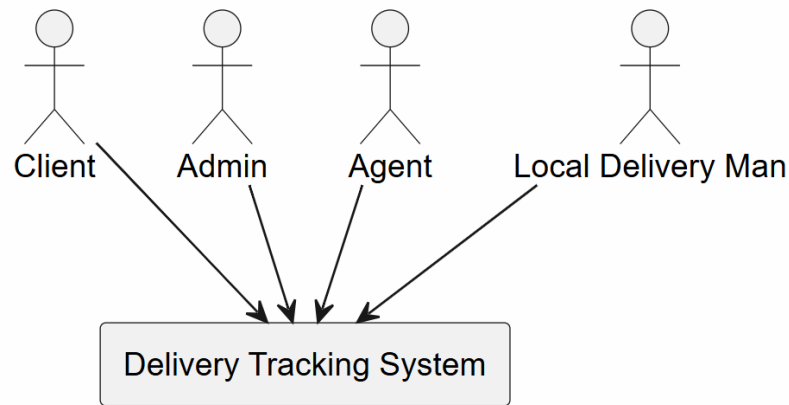


Figure 1: Static Context Diagram

2.3.1.2 Use Case Diagram — Client

Figure 2 shows the use case diagram for the Client actor. The client can register, log in, track parcels (by code or phone number), choose payment methods, and receive notifications. All account-dependent actions include the s'authentifier dependency.

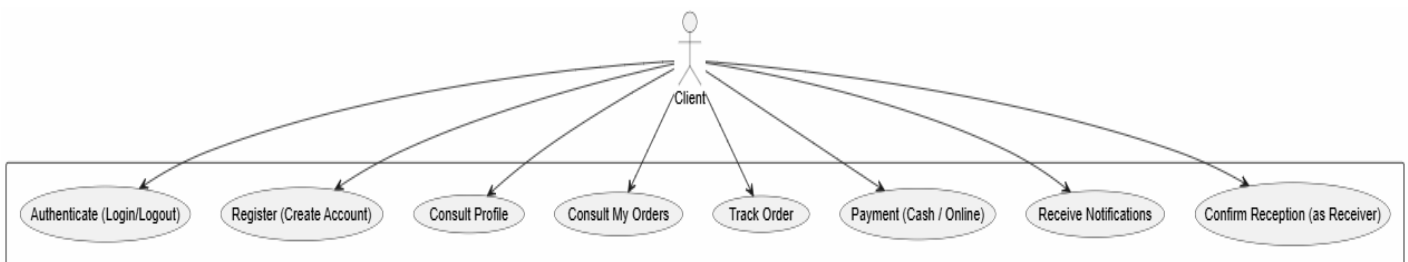


Figure 2: Use Case Diagram — Client

2.3.1.3 Use Case Diagram — Agent

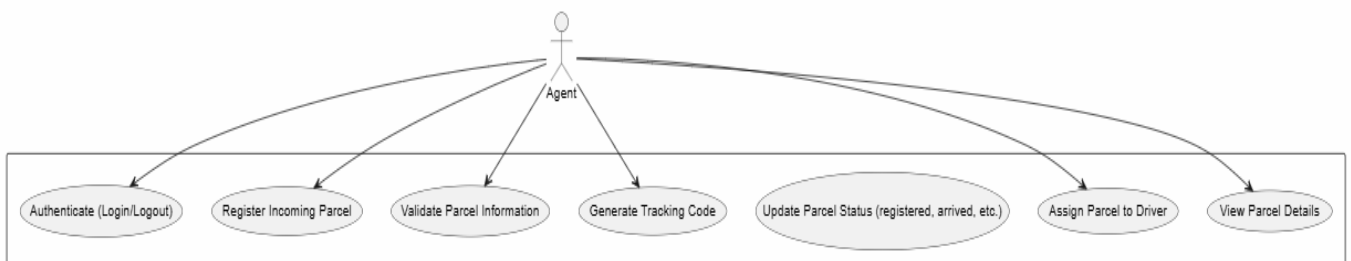


Figure 3: Use Case Diagram — Agent

Figure 3 shows the use case diagram for the Agent actor. The agent registers parcels, validates information, prints receipts, confirms inter-wilaya arrivals, and assigns drivers. All actions require authentication with a Staff ID.

2.3.1.4 Use Case Diagram — Driver

Figure 4 shows the use case diagram for the Driver actor. The driver views assigned parcels, accepts or refuses the list, starts deliveries, marks deliveries as delivered or failed, and handles COD payments

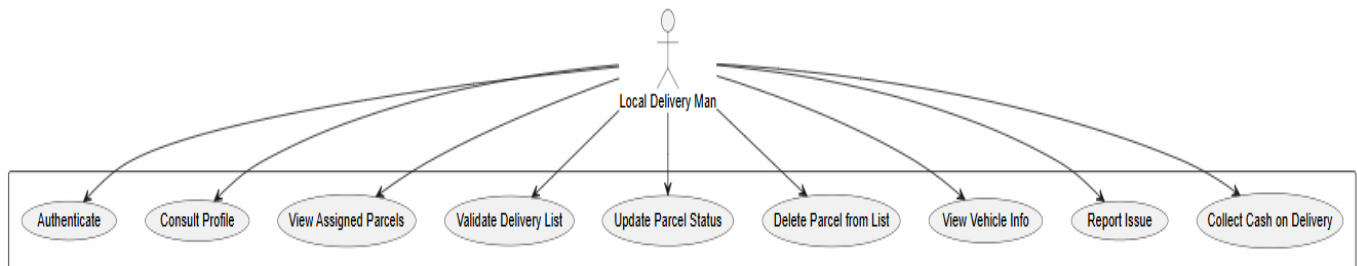


Figure 4: Use Case Diagram — Local Delivery Man

2.3.1.5 Use Case Diagram — Admin

Figure 5 shows the use case diagram for the Admin actor. The admin manages staff accounts and vehicles, monitors deliveries, generates reports, and configures system-level settings. All actions include authentication.

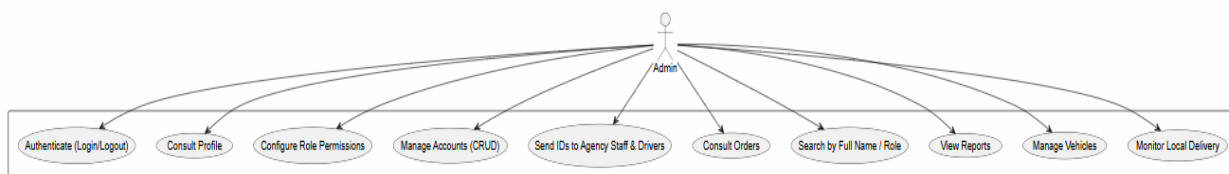


Figure 5: Use Case Diagram — Admin

2.4 Sequence Diagrams

Sequence diagrams show the interactions between actors and the system over time. They illustrate the dynamic behavior of the system for specific use cases. Below are the most important sequence diagrams for DeliverIT.

Each diagram follows RUP notation: actors on the left, the system lifeline, and alternative fragments (alt) for different scenarios.

2.4.1 Client Registration Sequence Diagram

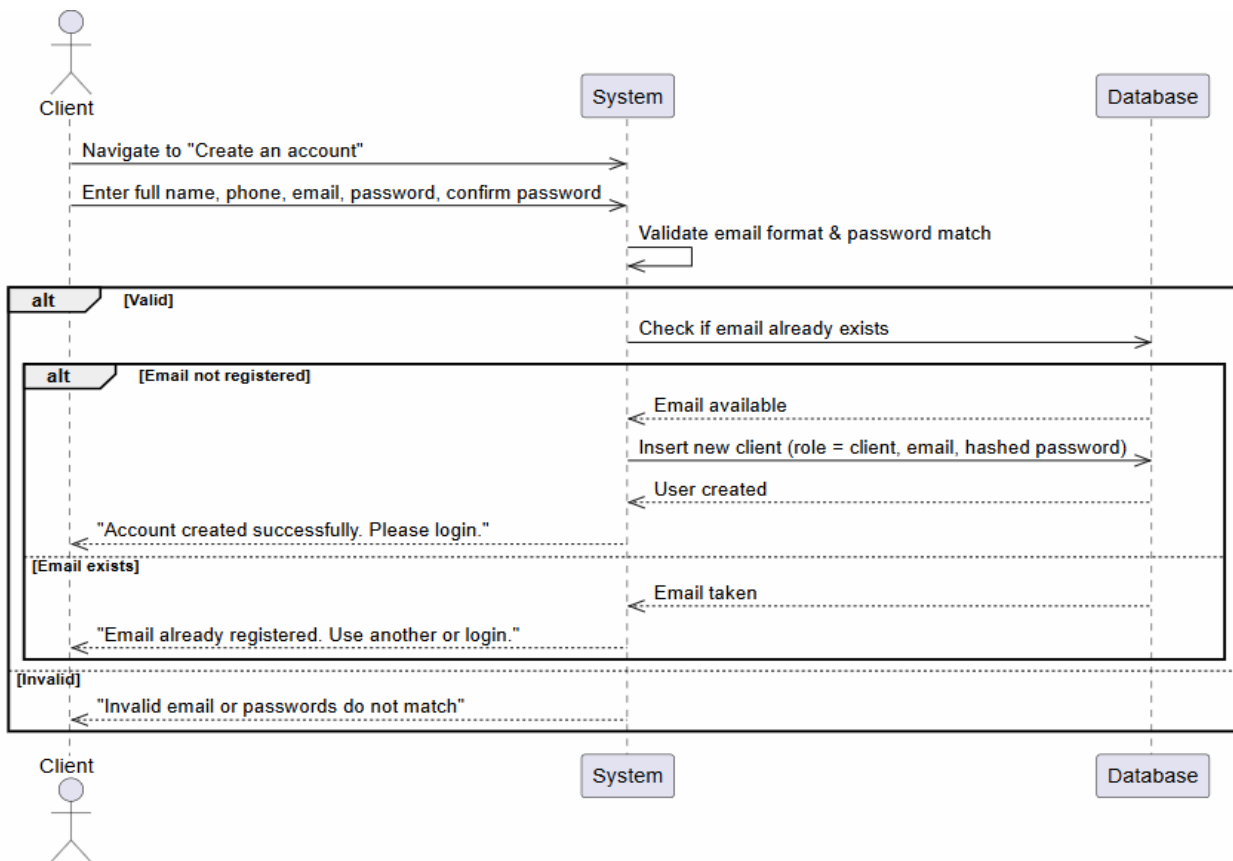


Figure 6: Client Registration Sequence Diagram

Description:

The client navigates to the registration page and enters their full name, phone number, email, password, and confirms the password. The system validates the email format and checks that the passwords match. If valid, the system checks the database to ensure the email is not already registered. If the email is available, the system creates a new user record with the role "client" and stores the hashed password. The system then returns a success message to the client. If the email already exists or the input is invalid, the system displays an appropriate error message.

2.4.2 Client Authentication Sequence Diagram

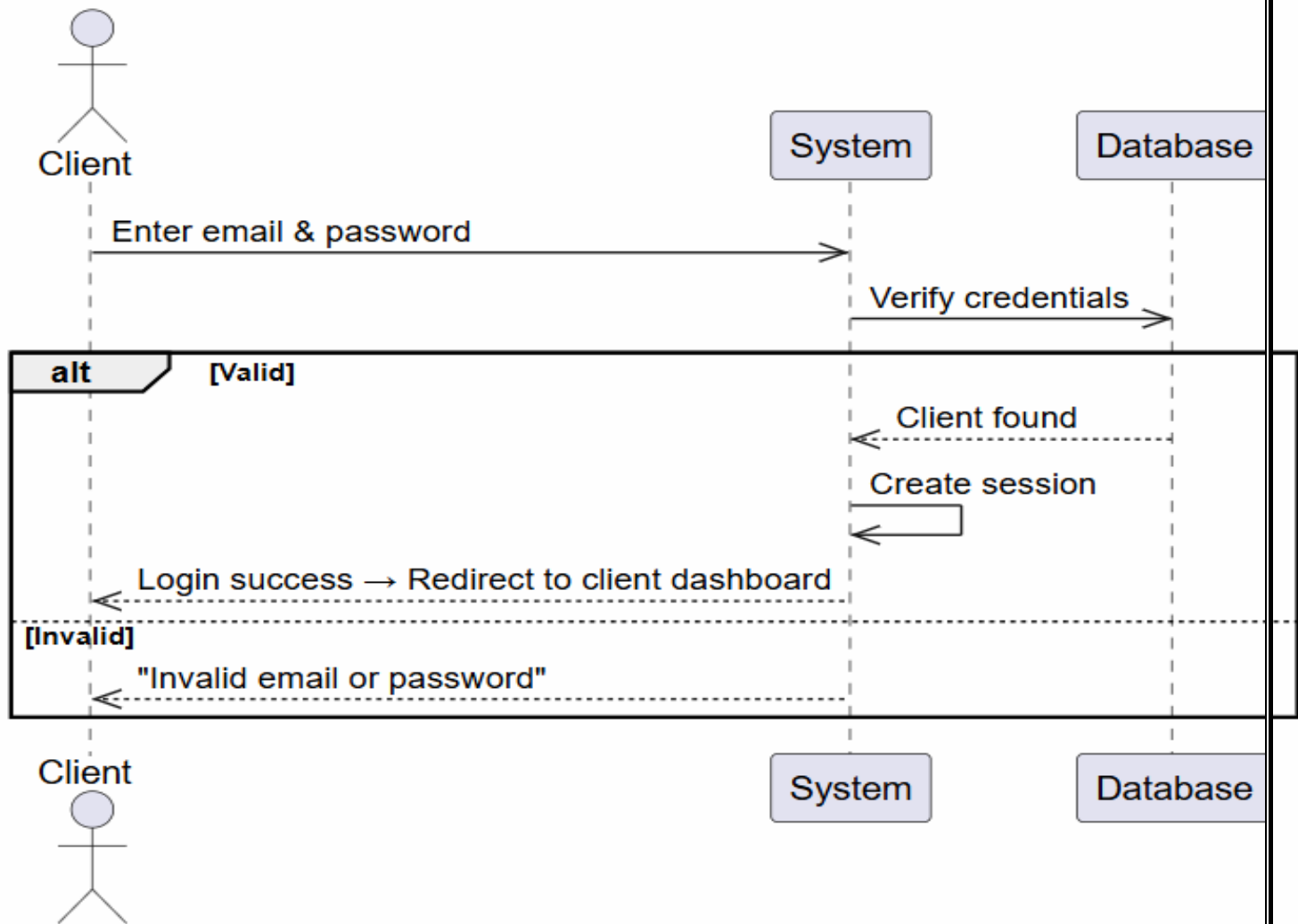


Figure 7: Client – Authentication seq

Description:

The client enters their email and password and optionally checks the "Remember me" box. The system sends a request to the database to verify the credentials. If the credentials are valid, the database returns the client's information with the role "client". The system creates a session and redirects the client to their dashboard. If the credentials are invalid, the system displays an error message "Invalid email or password".

2.4.3 Agency Staff Login Sequence Diagram

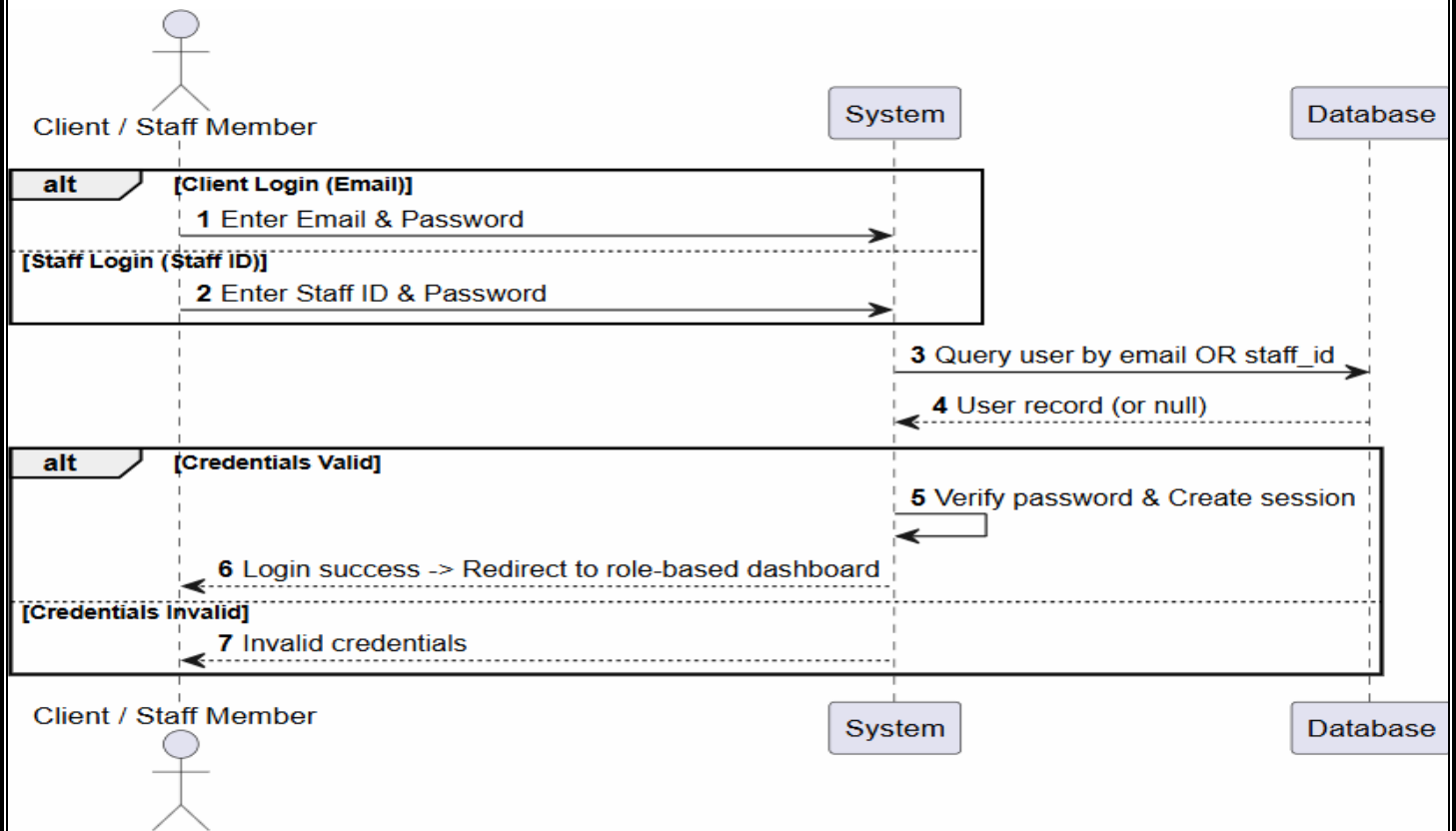


Figure 8: Auth Flow (Client & Staff Login) seq

Description:

The unified authentication flow handles both client login (using email and password) and staff login (using Staff ID and password). The user selects their login type. For clients, the system queries the database by email; for staff, it queries by Staff ID. The system verifies the password and creates a session with the appropriate role-based permissions. If the credentials are valid, the user is redirected to their role-specific dashboard. If invalid, an error message is displayed

2.4.4 Forgot Password Sequence Diagram

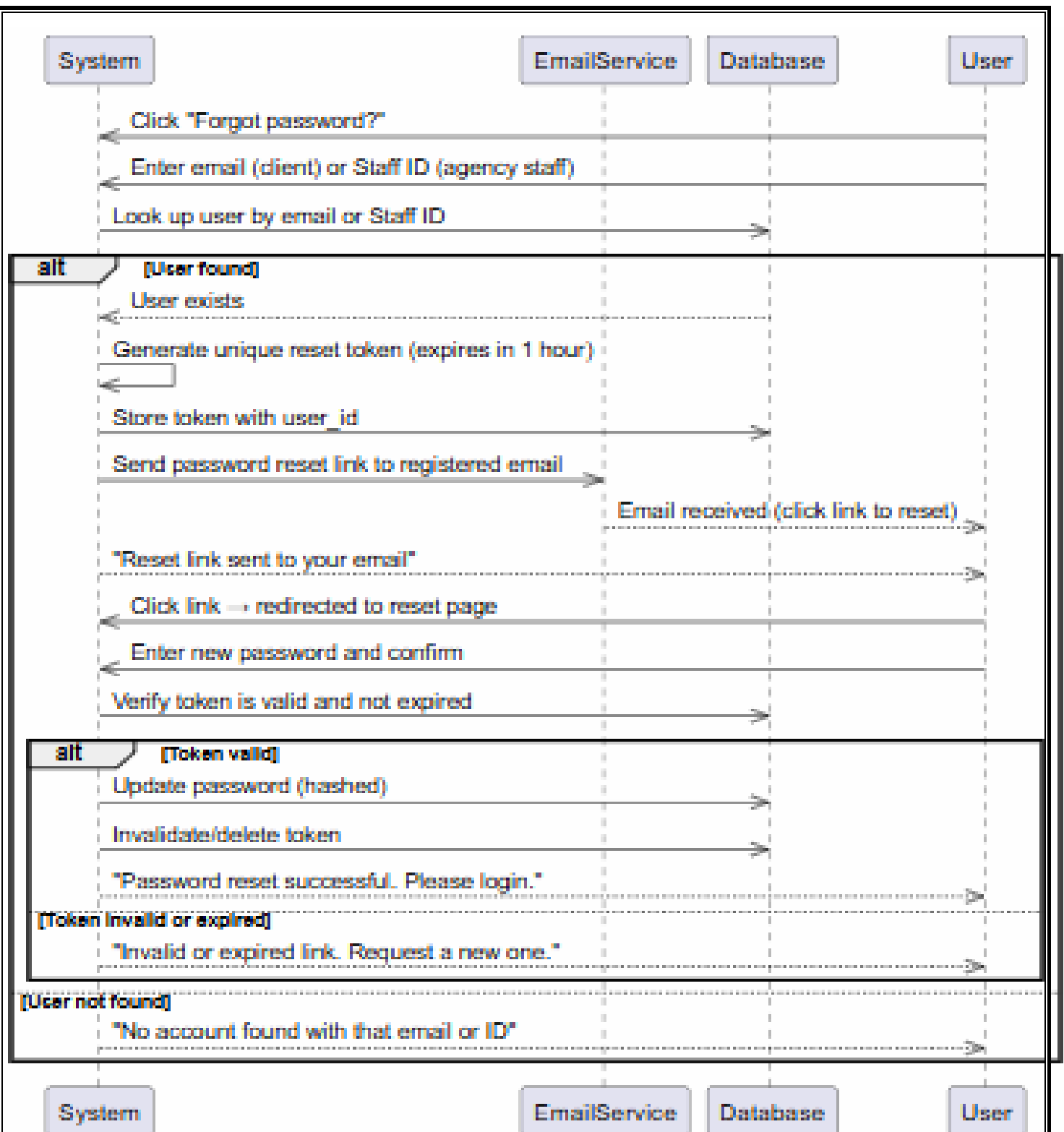


Figure 9: Forgot Password seq

2.4.5 User Management (Admin) Sequence Diagram

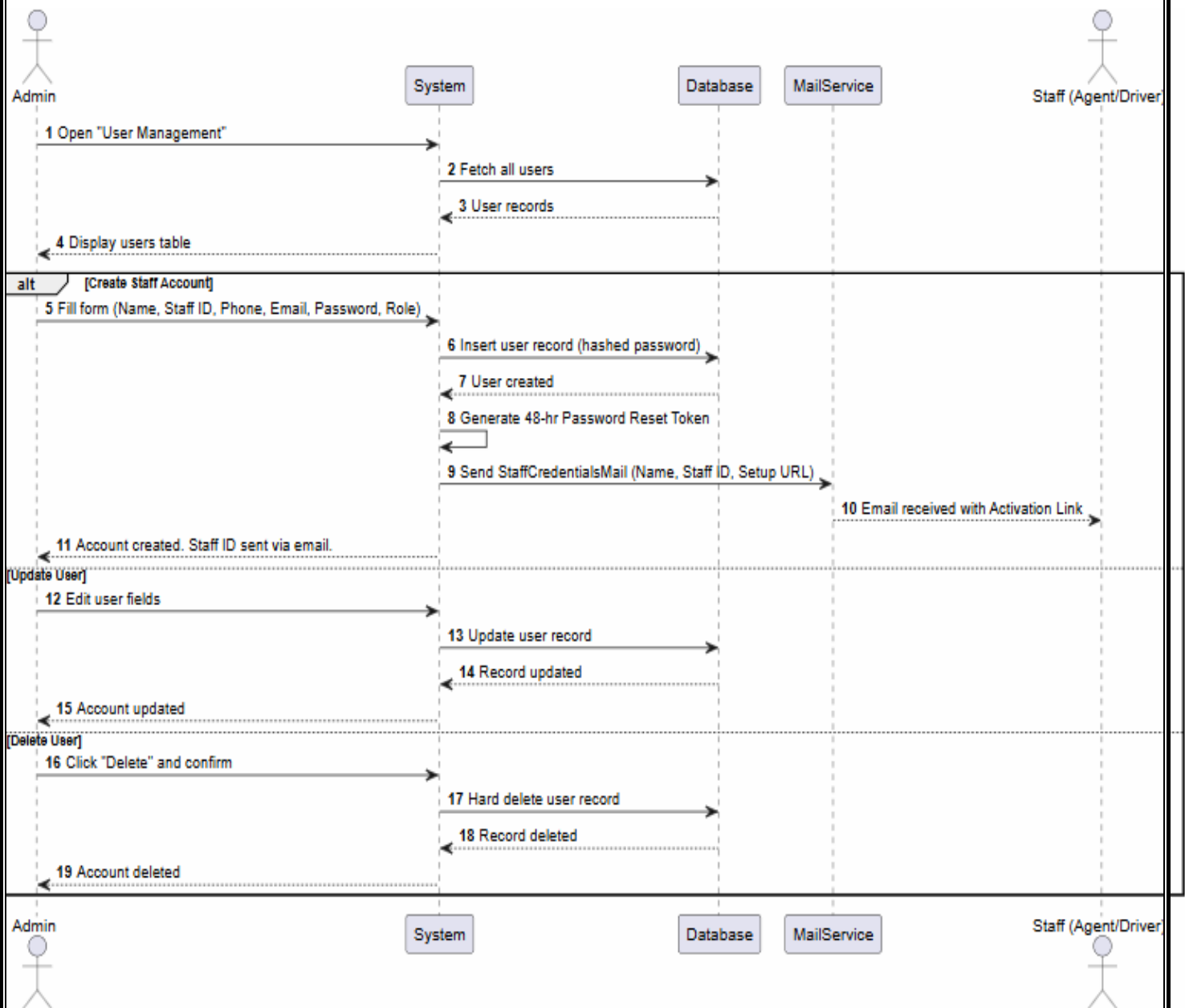


Figure 10: User Management (Admin) seq

Description:

The Admin opens the user management page. The system fetches all users from the database and displays the user table. The Admin can perform three types of operations: create, update, or delete a staff account. For creation, the Admin fills a form with the staff member's name, email, phone, role, and manually entered Staff ID.

The system inserts the new user record with a hashed password, generates a password reset token, and sends a welcome email containing the Staff ID and a setup link. The staff member receives the email and can activate their account. For updates, the Admin edits the user's fields, and the system updates the database record. For deletion, the Admin confirms the action, and the system hard deletes the user record from the database.

2.4.6 Profile Management Sequence Diagram

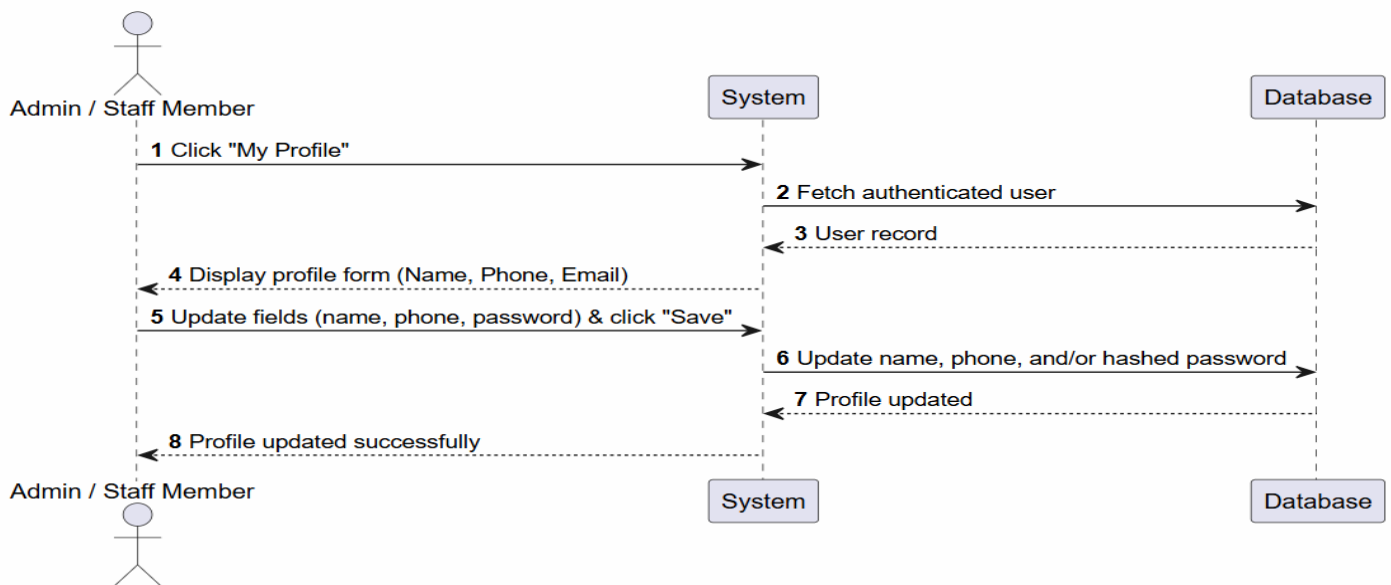


Figure 11: Profile Management seq

Description:

The staff member (Admin, Agent, or Driver) clicks "My Profile". The system fetches the authenticated user's data from the database and displays a profile form with fields for name, phone number, and password. The user updates the desired fields and clicks "Save Changes". The system sends a request to the database to update the name, phone number, and/or hashed password. After the database confirms the update, the system returns a success message to the user and updates the local storage or session.

2.4.7 Parcels Management & Search Sequence Diagram

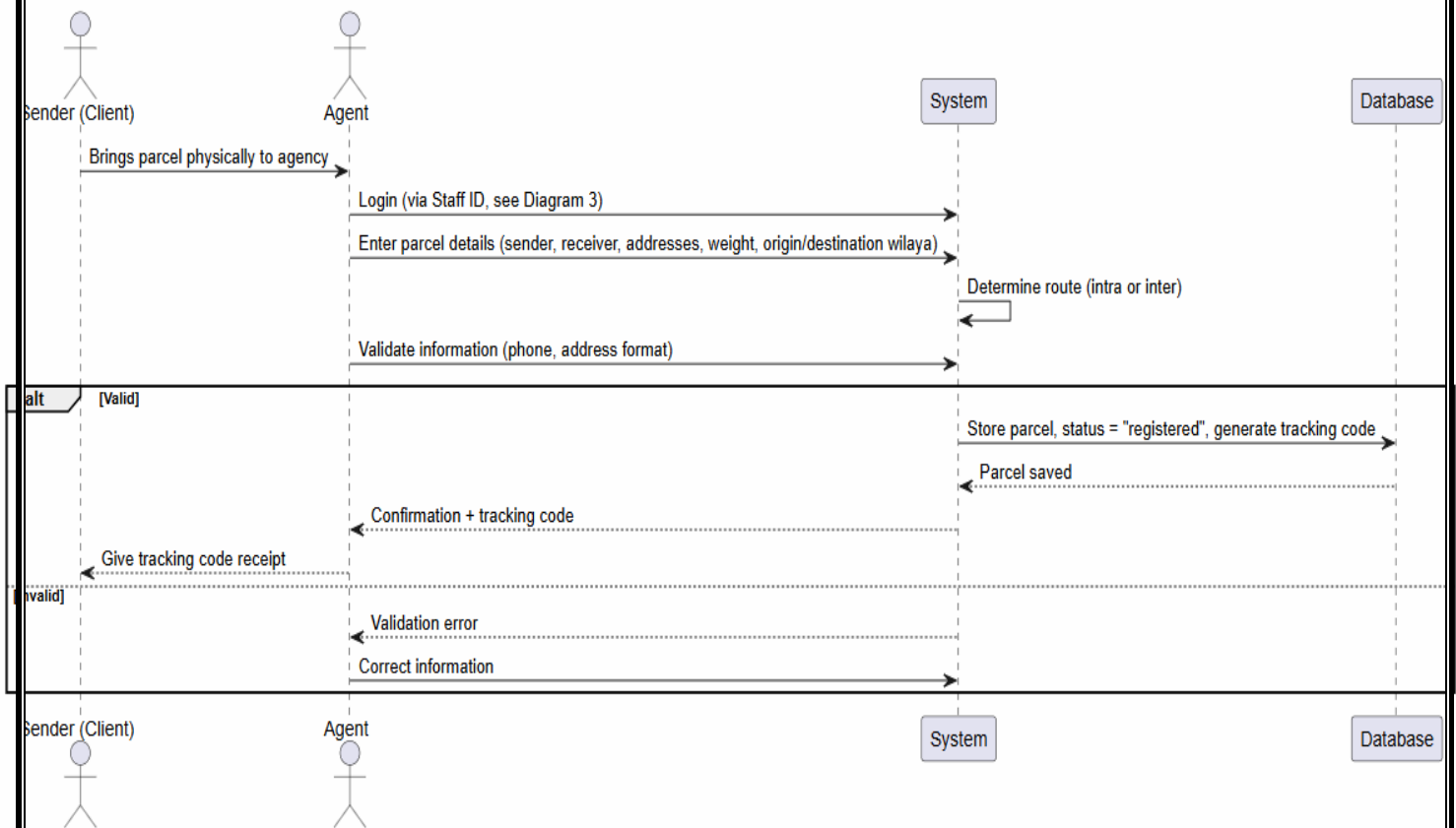


Figure 12: Parcels Management & Search seq

Description:

The Admin navigates to the "All Parcels" page. The system fetches all parcel records from the database and displays them in a table with status filters.

The Admin enters a search query (tracking code, sender name, receiver name, or destination). The system applies client-side filters based on the search query and the selected status dropdown, then displays the filtered parcel rows to the Admin.

2.4.8 Parcel Submission & Validation (Agent) Sequence Diagram

Figure 13: Parcel Submission & Validation (Agent) seq

Description:

The sender brings the parcel physically to the agency. The Agent logs in using their Staff ID and enters the parcel details: sender information, receiver information, address, weight, origin wilaya, and destination wilaya. The system determines whether the route is intra-wilaya (same wilaya) or inter-wilaya (different wilayas). The Agent validates the information (phone number format, address completeness). If valid, the system stores the parcel in the database with the status "registered", generates a unique tracking code, and returns a confirmation to the Agent. The Agent then gives the tracking code receipt to the sender. If the information is invalid, the system displays a validation error, and the Agent corrects the information.

Figure 14: Inter-Wilaya Transfer & Arrival Notification seq

2.4.9 Client Track Order Sequence Diagram

Description:

The client can track a parcel using either a tracking code or a phone number. If using a tracking code, the client enters the code on the public tracking page. The system fetches the parcel from the database. If the parcel is found and its status is "at_destination_agency" or later (assigned, out_for_delivery, delivered, failed), the system displays the current status, history, driver name, and vehicle information. If the status is "registered" or "in_transfer", the system displays "Tracking not yet available". If the tracking code is not found, the system displays an error. If using a phone number, the client enters their phone number. The system finds all orders associated with that phone number and displays a list. The client selects a specific order, and the system fetches and displays detailed tracking information. If no orders are found, the system displays "No orders found with this phone number."

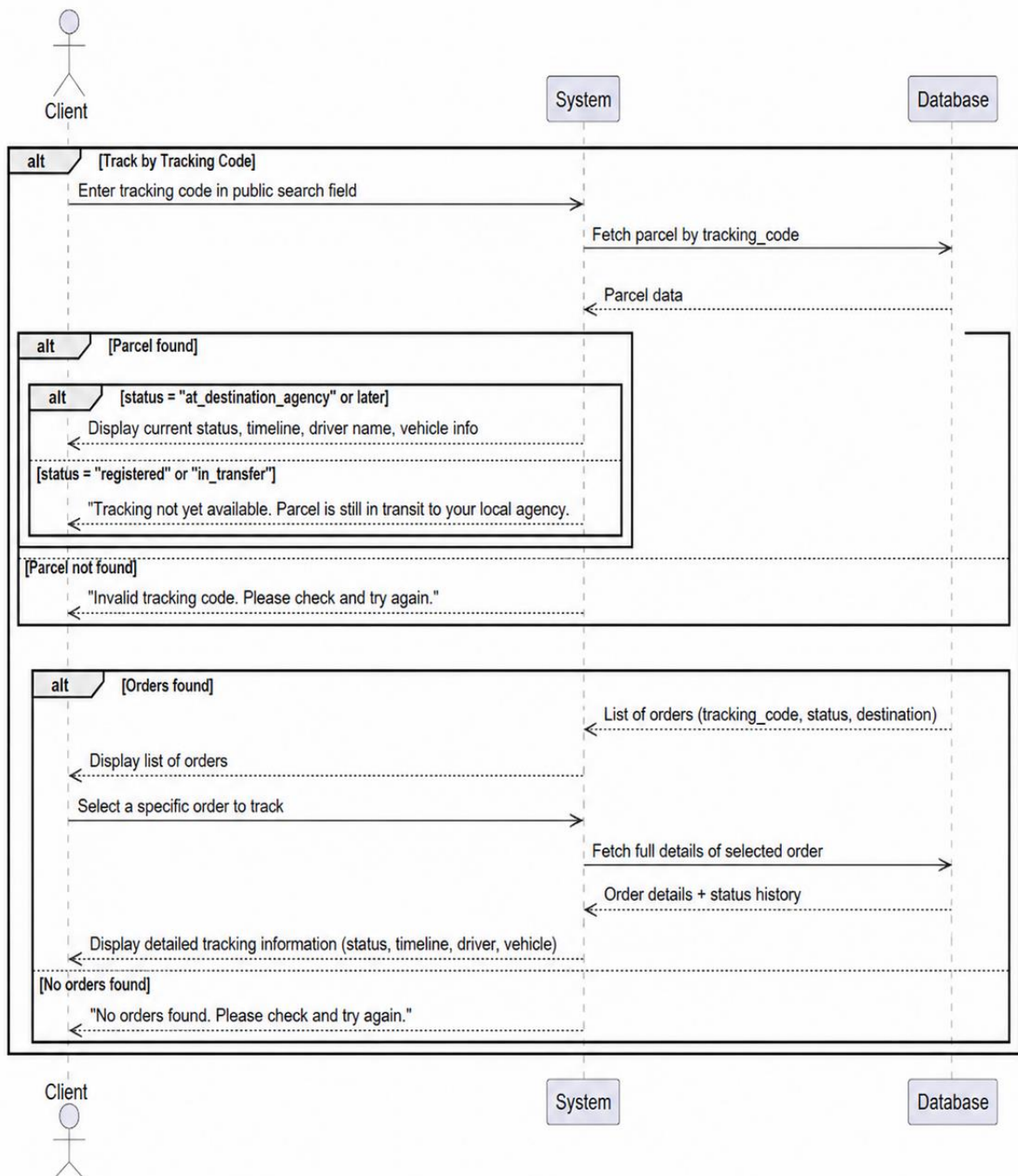


Figure 15: Client – Track Order seq

2.4.10 Driver Authentication Sequence Diagram

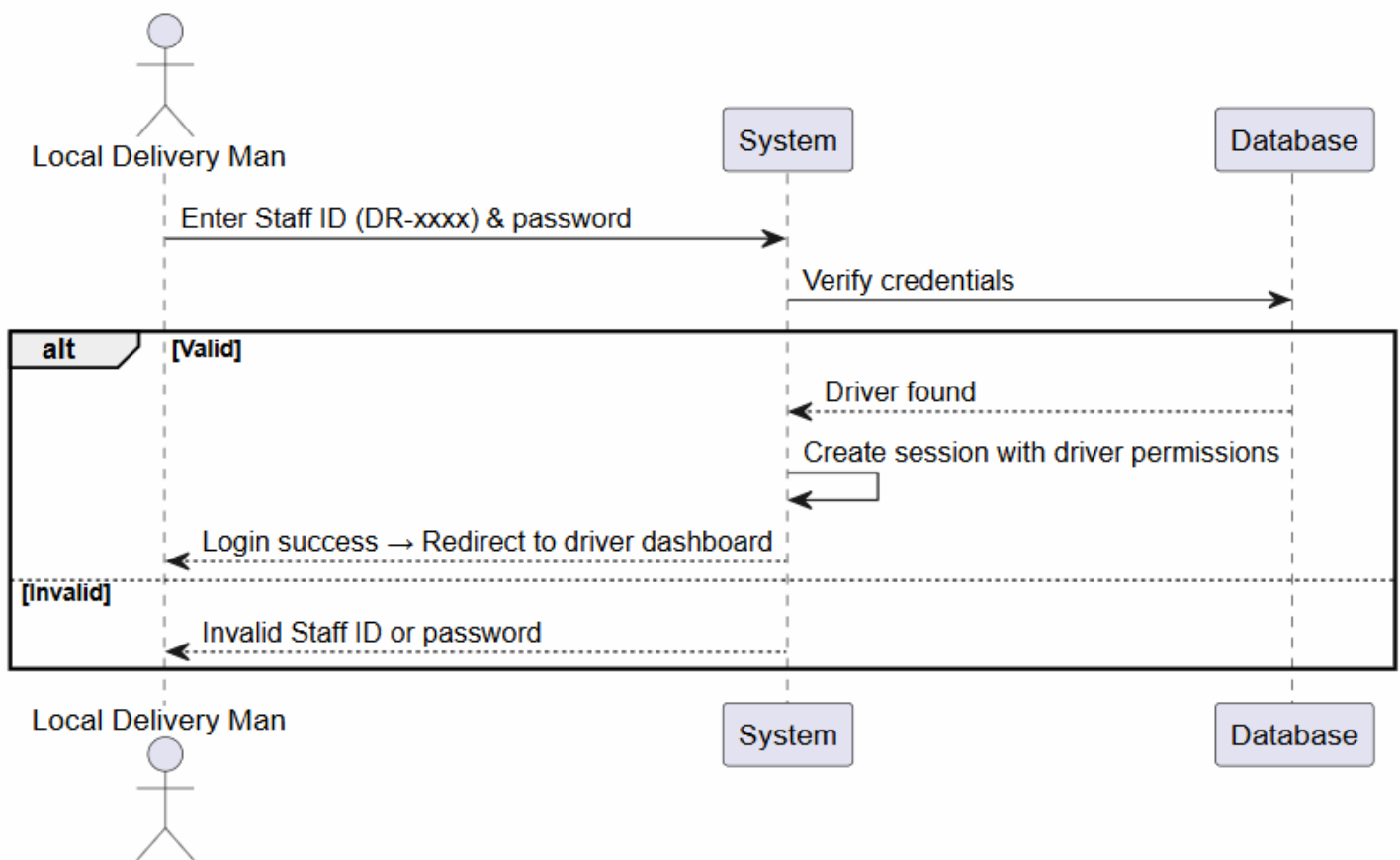


Figure 16: Driver – Authentication seq

Description:

The Driver enters their Staff ID (with the DR- prefix) and password. The system identifies the role as "driver" from the prefix and verifies the credentials against the database. If the credentials are valid, the system creates a session with driver-specific permissions and redirects the Driver to their dashboard. If invalid, the system displays "Invalid Staff ID or password".

2.4.12 Driver Consult Profile Sequence Diagram

Figure 17: Driver – Consult Profile seq

Description:

The Driver clicks "My Profile". The system fetches the driver's profile data along with their assigned vehicle. The system then displays the profile information (name, phone, email) and vehicle details to the Driver.

2.4.11 Driver Updates Status Sequence Diagram

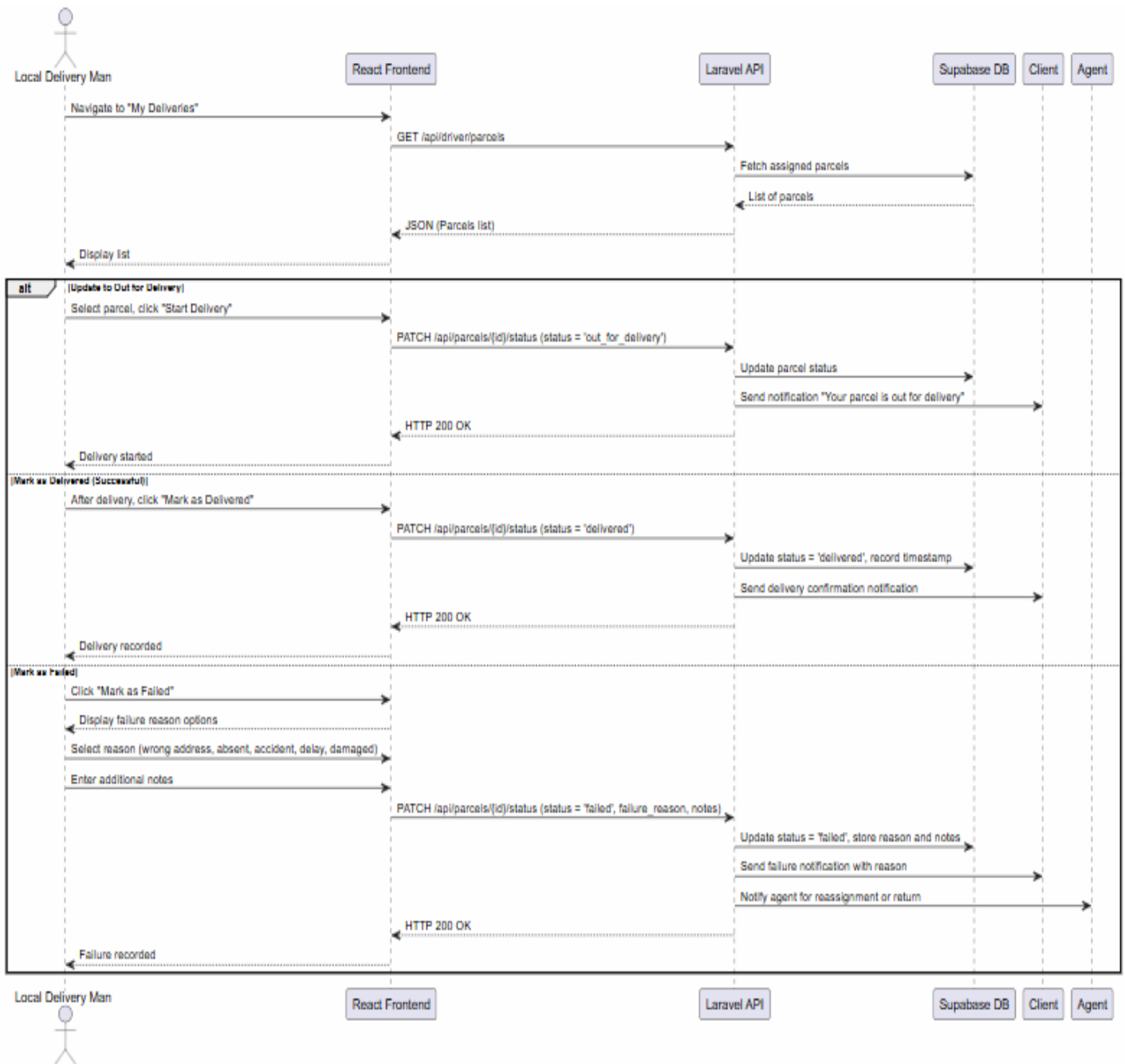


Figure 18: Driver Updates Status seq

Description:

The Driver navigates to "My Deliveries". The system fetches and displays the list of assigned parcels. The Driver can perform three types of actions. For starting a delivery, the Driver selects a parcel and clicks "Start Delivery". The system updates the status to "out_for_delivery"

For a successful delivery, the Driver clicks "Mark as Delivered". The system updates the status to "delivered", records a timestamp, and sends a delivery confirmation notification to the client. For a failed delivery, the Driver clicks "Mark as Failed", selects a reason from a list, and enters additional notes. The system updates the status to "failed_delivery", stores the reason and notes, sends a failure notification to the client, and notifies the Agent for reassignment or return.

2.4.12 Parcel Assignment and Delivery Sequence Diagram

Description:

This comprehensive diagram covers the entire parcel lifecycle from registration to confirmation. In phase 1 (Registration), the Agent registers a parcel with sender, receiver, wilayas, weight, and payment information. The system saves the parcel and sends an SMS to the receiver with the tracking link and code. In phase 2 (Assignment), the Agent selects a driver and assigns the parcel. The system updates the status to "assigned". In phase 3 (Driver Response), the Driver opens their assigned list. If the Driver accepts, the status becomes "accepted". If the Driver refuses, the status returns to "registered" and the Agent is notified for reassignment. In phase 4 (Delivery), the Driver sets the status to "out_for_delivery", then to "delivered" upon arrival. The client receives SMS notifications at each step. In phase 5 (Confirmation), the client opens the tracking page, clicks "Confirm Reception", and leaves a rating and feedback. The system updates the status to "confirmed".

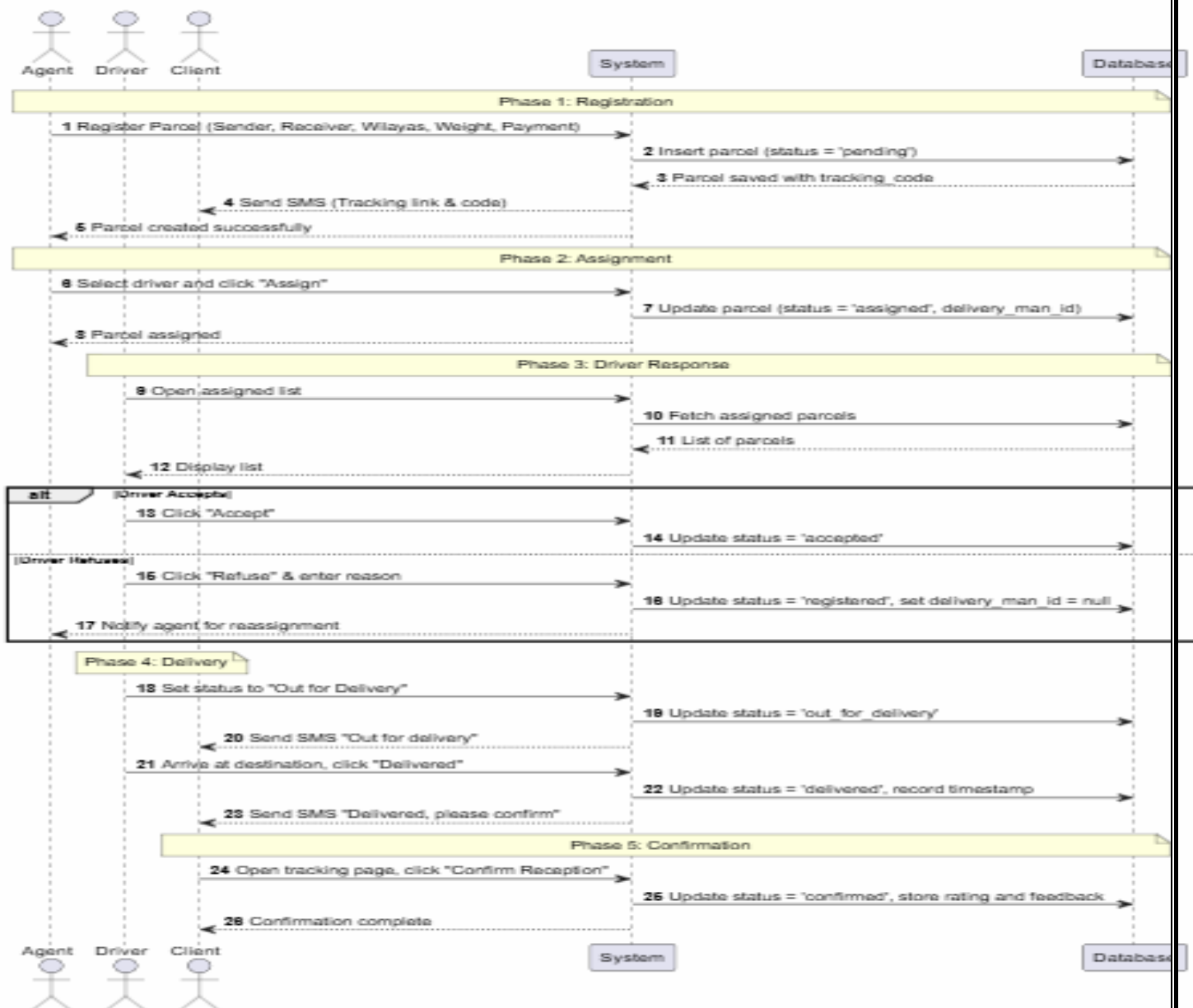


Figure 19: Parcel Assignment and Delivery seq

2.4.13 Payment – Online (upload proof image) Sequence Diagram

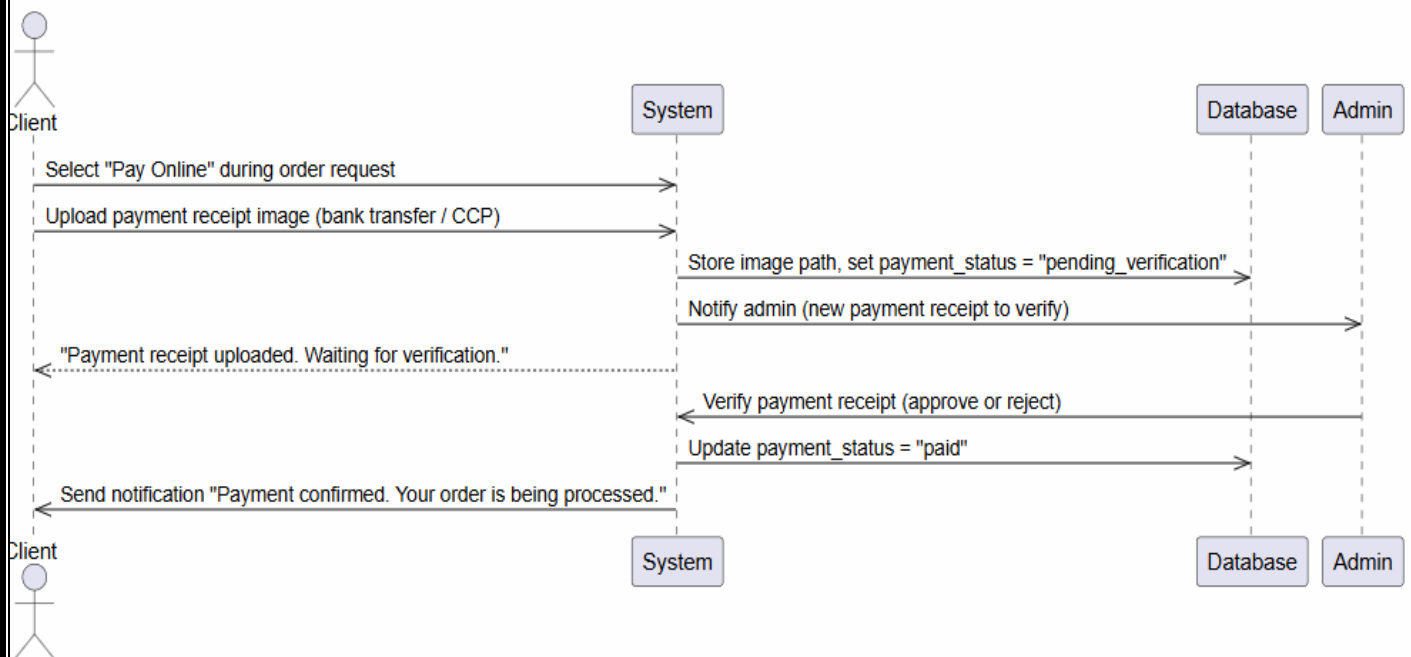


Figure 21: Payment – Online (upload proof image) seq

Description:

During the order request, the Client selects "Pay Online". The Client uploads a payment receipt image (bank transfer or CCP). The system stores the image path and sets the payment status to "pending_verification". The system notifies the Admin that a new payment receipt is waiting for verification. The Client is informed that their payment receipt has been uploaded and is awaiting verification. The Admin then verifies the receipt. If approved, the Admin updates the payment status to "paid", and the system sends a confirmation notification to the client. If rejected, the Admin enters a reason, and the system notifies the client to upload a valid receipt.

2.4.14 Exception Handling by Agent Sequence Diagram

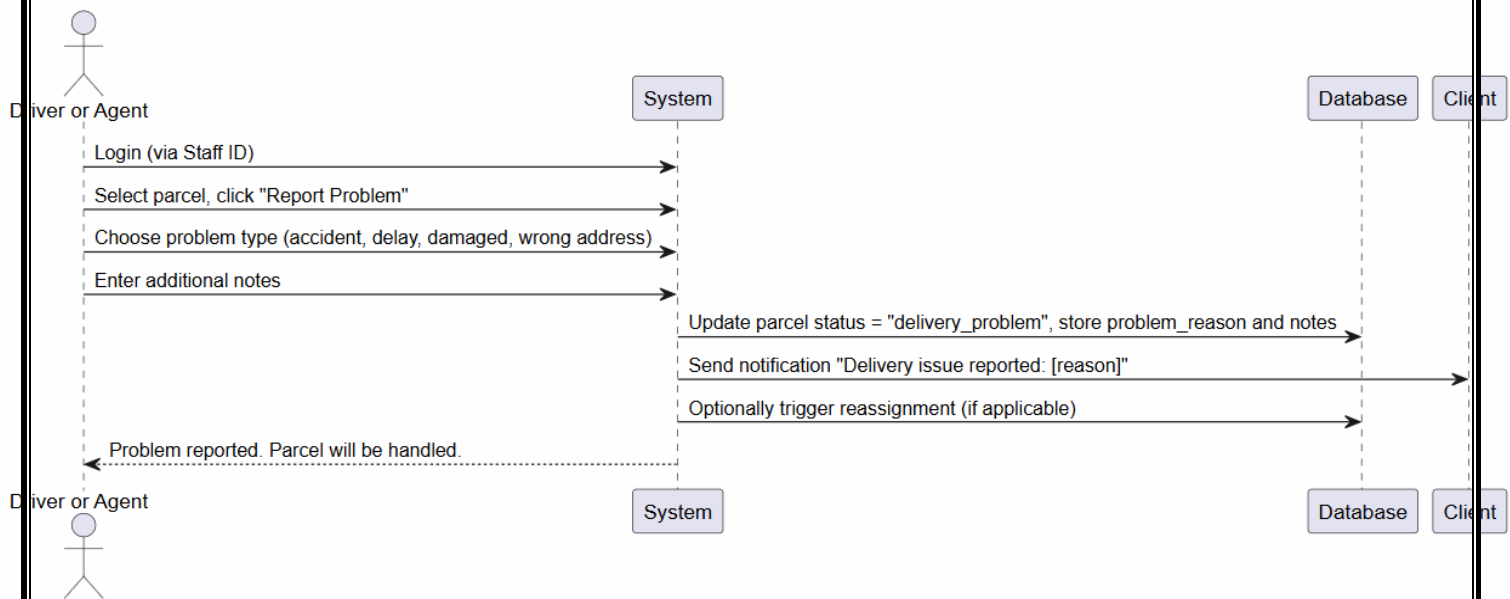


Figure 22: Exception Handling by Agent seq

Description:

The Driver or Agent logs in using their Staff ID. They select a parcel and click "Report Problem". They choose a problem type (accident, delay, damaged, wrong address) and enter additional notes. The system updates the parcel status to "delivery_problem", stores the problem reason and notes, and sends a notification to the client: "Delivery issue reported: [reason]". The system optionally triggers reassignment if applicable. Finally, the system confirms to the user that the problem has been reported and the parcel will be handled.

2.4.15 Admin Consult Profile Sequence Diagram

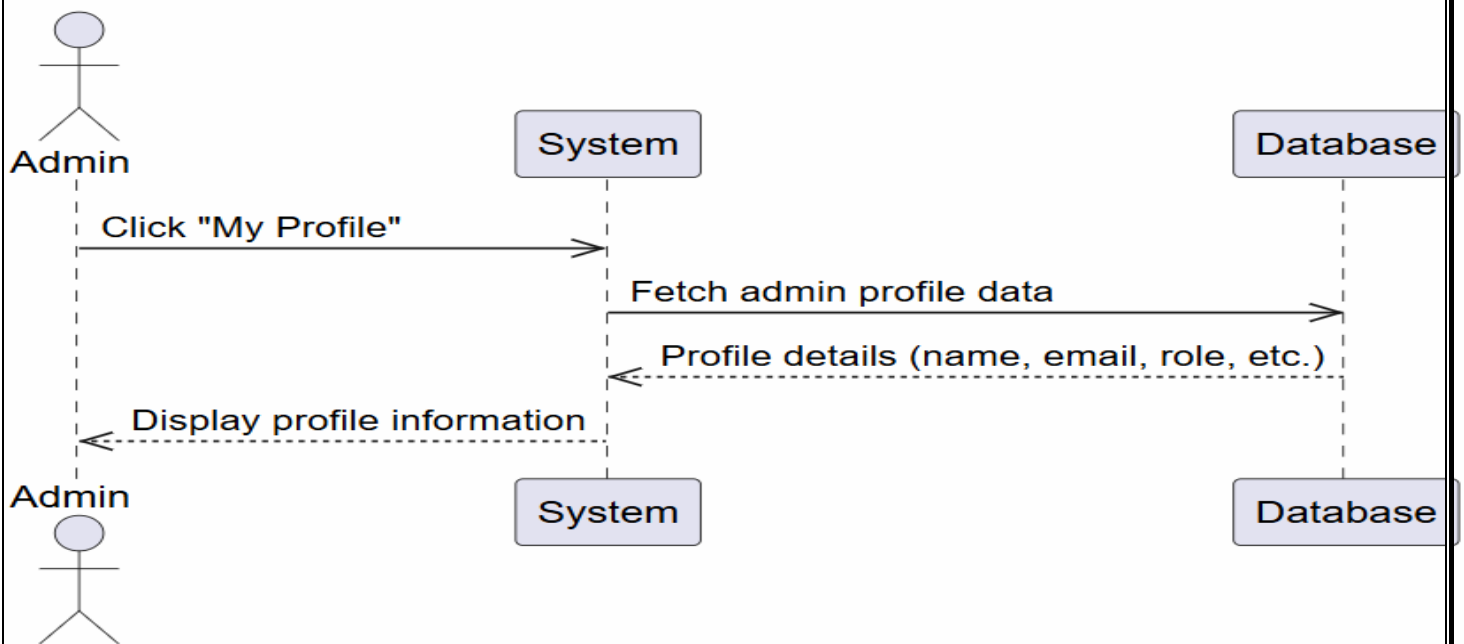


Figure 23: Admin – Consult Profile seq

Description:

The Admin clicks "Their Name". The system fetches the Admin's profile data (name, email, role, etc.) from the database and displays the profile information.

2.5 Class Diagram

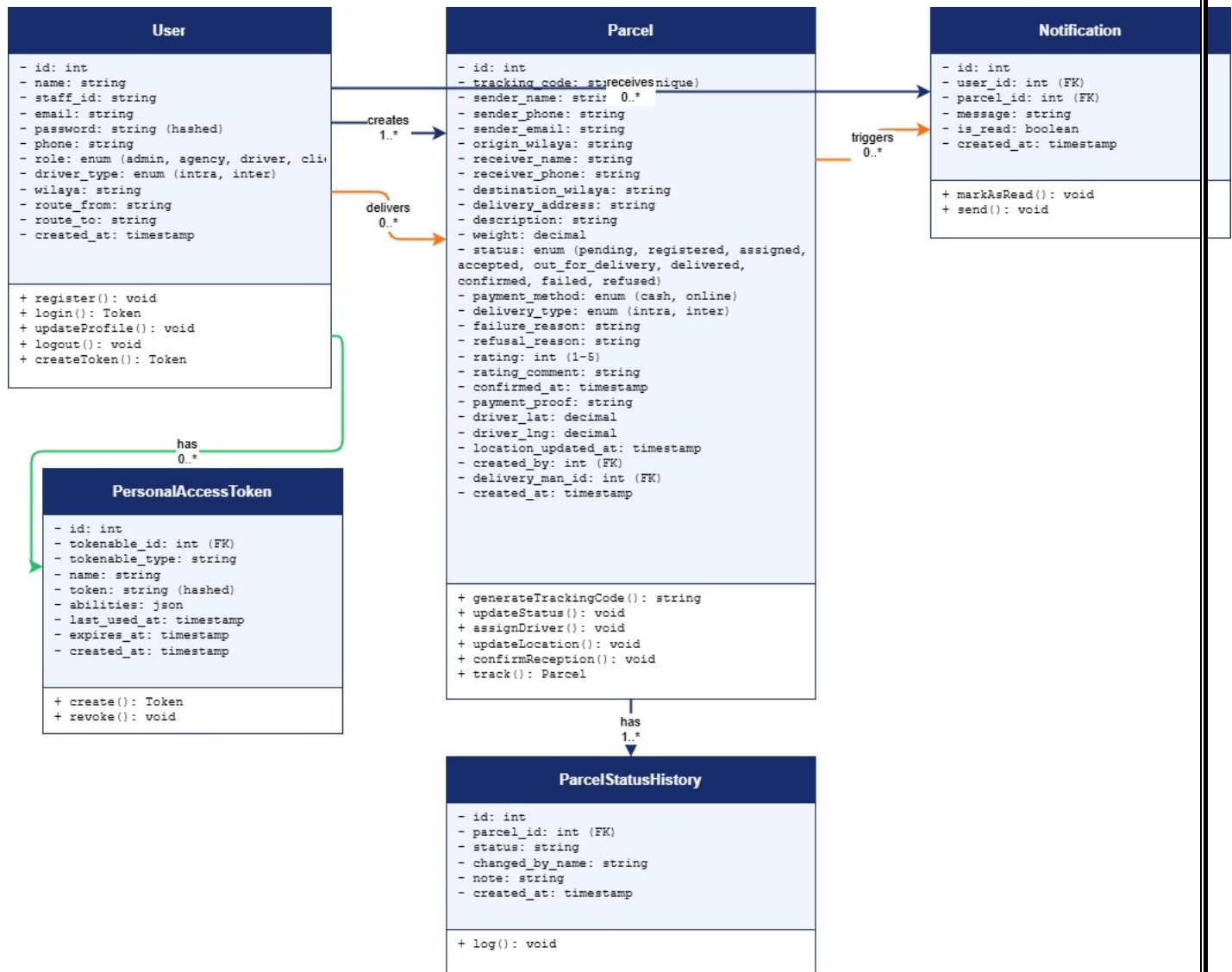


Figure 24: class diag

Description

The DeliverIt system is structured around four main classes: User, Parcel, ParcelStatusHistory, and Notification, plus PersonalAccessToken managed by Laravel Sanctum.

User Represents all system actors in a single class differentiated by the role attribute. It stores the user's name, email, phone, and hashed password. For staff members it includes a staff_id following the format AD-001, AG-001, or DR-001. The role attribute takes one of four values: admin, agency, driver, or client. For drivers, additional fields are stored: driver_type (intra or inter), wilaya for intra-wilaya drivers, and route_from and route_to for inter-wilaya drivers. Its main methods are register(), login(), updateProfile(), logout(), and createToken().

Parcel The central entity of the system. It holds all information related to a delivery: sender and receiver details including name, phone, wilaya, and commune, the delivery address shown to the driver, the auto-generated tracking code, the parcel description and weight, and the payment method (cash or online). The status attribute tracks the full lifecycle: pending → registered → assigned →

accepted → out_for_delivery → delivered → confirmed, with two alternative endings: failed and refused. It also stores the failure_reason and refusal_reason reported by the driver, the payment_proof file path for online payments, the driver's live GPS coordinates (driver_lat and driver_lng updated every 5 seconds), and the client's rating and comment after delivery. Two foreign keys link the parcel to its registering agent (created_by) and its assigned driver (delivery_man_id). Its main methods are generateTrackingCode(), updateStatus(), assignDriver(), updateLocation(), confirmReception(), and track().

ParcelStatusHistory Records every status change a parcel undergoes. Each record stores the parcel_id as a foreign key, the new status value, the name of the person who triggered the change (changed_by_name), an optional note, and the exact timestamp. This creates a complete audit trail for every parcel. Its only method is log(), which is called automatically every time a parcel status changes.

Notification Stores in-app alerts sent to agents when a driver refuses an assigned parcel. Each notification holds the user_id of the receiving agent, the parcel_id of the related parcel, the message text containing the driver name, tracking code, and refusal reason, and a boolean is_read flag defaulting to false. Its methods are send() and markAsRead().

Relationships

- A User (agent) creates zero or more Parcels — each parcel has exactly one registering agent.
- A User (driver) is assigned to zero or more Parcels — each parcel has at most one assigned driver.
- A Parcel has one or more ParcelStatusHistory records — one created at registration and one added at every status change.
- A User receives zero or more Notifications — each notification belongs to exactly one user.
- A Parcel triggers zero or more Notifications — specifically when a driver refuses an assignment.
- A User has zero or more PersonalAccessTokens — one created at each login and revoked at logout.

2.6 State Diagram

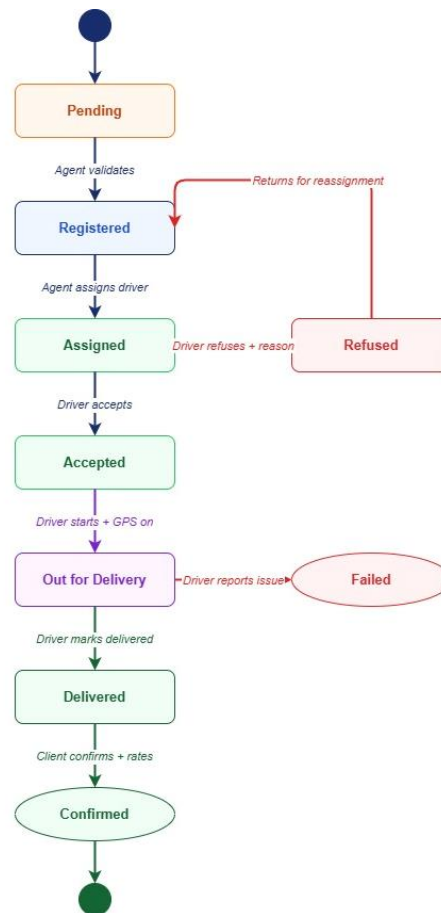


Figure 25: state diag

The DeliverIt system defines a clear and linear state machine for every parcel. A parcel passes through the following states from creation to final resolution:

- **Initial State:** Pending The parcel enters this state the moment the agent fills in the registration form. It is waiting for the agent to review and validate the information.
- **Registered:** The agent has validated all parcel details. The parcel is now confirmed in the system and waiting to be assigned to a driver.
- **Assigned :**The agent has selected a compatible driver (intra or inter-wilaya) and assigned the parcel. The driver has been notified.
- **Accepted:** The driver has reviewed the parcel details and accepted the delivery mission. The parcel is ready to be picked up.
- **Out for Delivery :**The driver has started the delivery. GPS sharing is active — the driver's location is sent to the server every 5 seconds and displayed on the client's tracking page in real time.
- **Delivered :**The driver has marked the parcel as successfully handed to the receiver. The client can now confirm reception and submit a rating.
- **Confirmed (Final):** The client has confirmed reception and optionally submitted a star rating. This is the successful terminal state.
- **Failed (Final) :**The driver reported that the delivery could not be completed, with a reason (address not found, recipient absent, etc.). This is a terminal failure state.
- **Refused :**back to Registered The driver refused the assignment before starting delivery, with a reason. The parcel returns to the Registered state so the agent can assign it to a different driver. This is not a terminal state.

2.7 Entity-Relationship Diagram (ERD)

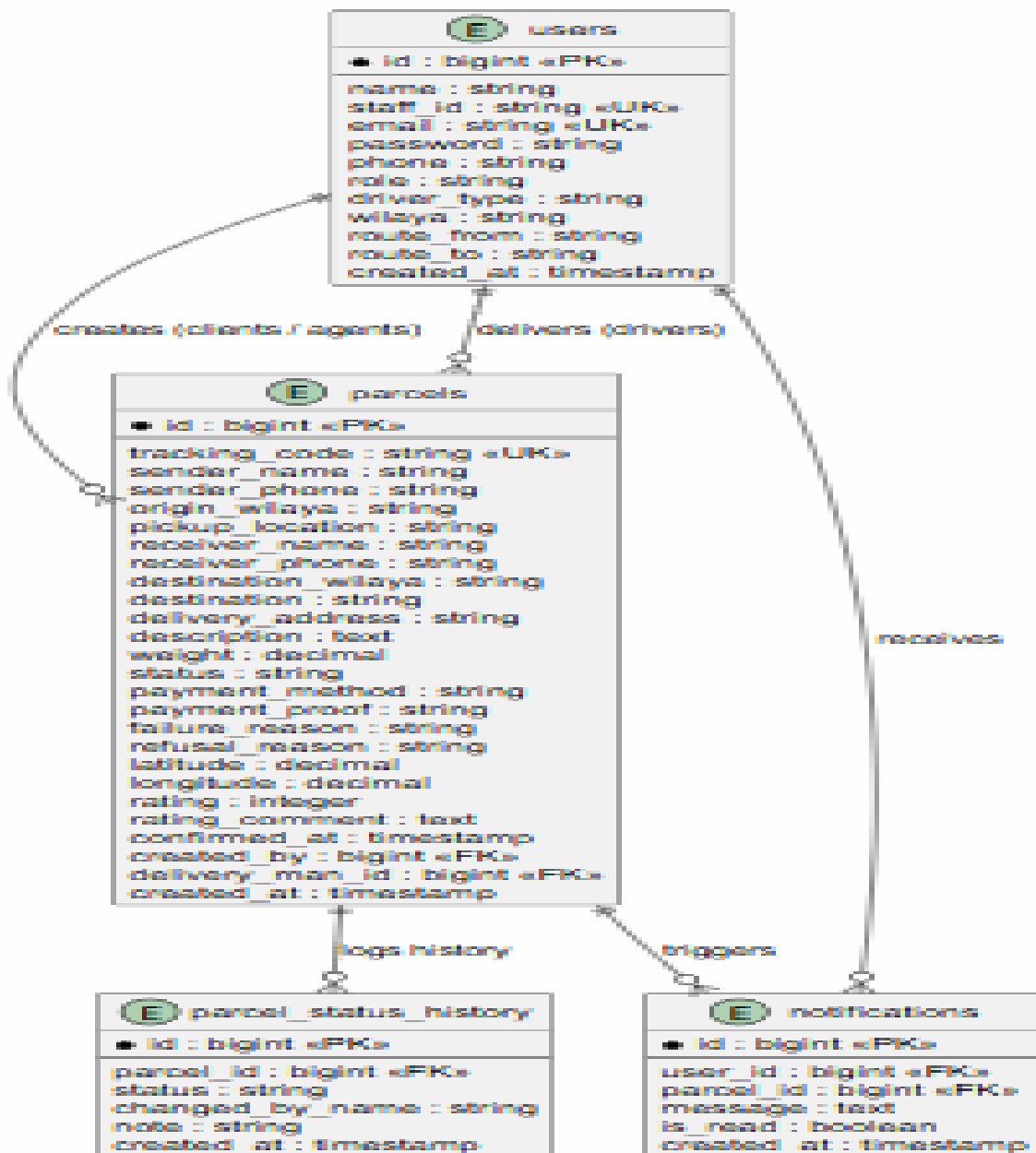


Figure 26: Entity-Relationship Diagram (ERD)

Description:

The Entity-Relationship Diagram (ERD) represents the logical structure of the database for the DeliverIT system. It shows the main entities (tables), their attributes, and the relationships between them.

The main entities are:

users: contains user information

(id, name, email, password, phone, role, staff_id, driver_type, wilaya, route_from, route_to, created_at).

parcels: contains parcel information

(id, tracking_code, sender_name, sender_phone, origin_wilaya, pickup_location, receiver_name, rec

eiver_phone, destination_wilaya, destination, delivery_address, description, weight, status, payment_method, payment_proof, failure_reason, refusal_reason, rating, rating_comment, confirmed_at, created_by, delivery_man_id, created_at).

parcel_status_history: logs all status changes for each parcel (id, parcel_id, status, changed_by_name, note, created_at).

notifications: stores notifications sent to users (id, user_id, parcel_id, message, is_read, created_at).

The relationships are:

A user creates parcels (users one-to-many parcels via created_by).

A user delivers parcels (users one-to-many parcels via delivery_man_id).

A parcel has many status history entries (parcels one-to-many parcel_status_history).

A user receives many notifications (users one-to-many notifications).

A parcel triggers many notifications (parcels one-to-many notifications).

This ERD provides the foundation for the database implementation in Chapter 3.

2.8 Conclusion

In this chapter, we performed the modeling of the DeliverIT – Delivery Tracking System using the UML language. We defined the interactions between the different actors (Admin, Agent, Driver, Client) and the system, allowing a better structuration of the project.

We presented:

The context diagram to delimit the system boundary.

Use case diagrams for each actor (Admin, Agent, Client, Driver) as well as a general use case diagram to provide an overview.

Sequence diagrams for all key scenarios, including registration, authentication, user management, parcel submission, inter-wilaya transfer, tracking, driver operations, payment (COD and online), and exception handling.

The class diagram to represent the static structure of the system.

The state diagram to model the lifecycle of a parcel.

The entity-relationship diagram (ERD) to define the database structure.

This modelling constitutes an essential foundation for the development phase, ensuring an implementation that conforms to the defined requirements and facilitating the integration of the planned functionalities.

In the next chapter, we will move to the concrete implementation of the DeliverIT system, using the technologies presented in the preliminary study (PHP, MySQL, HTML, CSS, JavaScript) and following the MVC architecture. We will present the main interfaces of the application, the database schema, and code extracts for the most important functionalities.

Chapter 3: System Implementation

3.1 Introduction:

This chapter presents the implementation phase of the DeliverIt delivery tracking system. It describes the development tools and programming languages used, explains the technical choices made during the design process, and presents the main interfaces of the application along with their functionalities.

3.2 Implementation Tools:

3.2.1 Visual Studio Code

Visual Studio Code (VS Code) was used as the primary code editor throughout the development process. It is a lightweight, fast, and highly extensible editor with built-in support for JavaScript, PHP, JSX, and JSON. Various extensions were installed to facilitate React and Laravel development.

3.2.2 Programming Languages and Frameworks

Frontend:

- React.js (Vite): A JavaScript library used to build dynamic and reactive user interfaces through reusable components. Vite was used as the build tool for fast development and optimized production builds.
- CSS: Used to style the components and maintain a consistent visual identity throughout the application, using CSS variables for the color scheme (navy #1a2e6e, orange #f97316).
- JavaScript (ES6+): Used for client-side logic, API communication, and state management.

Backend:

- PHP / Laravel: A PHP framework used to build a RESTful API. It handles authentication, business logic, routing, and database communication.
- Laravel Sanctum: Used for token-based authentication, allowing secure access to protected API routes.

3.2.3 Database — Supabase (PostgreSQL)

Supabase is a cloud-based platform built on PostgreSQL. It hosts the project database and provides secure remote access. The main tables in the database are: users, parcels, parcel_status_history, and notifications.

3.2.4 Additional Tools

- Laragon: A local development environment for Windows that provides a PHP server and MySQL database for local testing.

- OpenStreetMap / Leaflet.js: An open-source mapping library used to display delivery routes and real-time GPS driver locations on an interactive map.
- Gmail SMTP: Used to send automated email notifications, including staff account credentials and parcel receipt confirmations.

3.2.5 Justification of Technical Choices

The selected technologies are free, well-documented, and widely adopted in the web development industry. The combination of React and Laravel represents a modern client-server architecture (SPA + REST API) that ensures:

- A smooth and responsive user interface.
- A secure and scalable API backend.
- A robust relational database structure.
- Easy maintenance and future extensibility.

3.3 Application Interfaces

3.3.1 Public Interface — Homepage

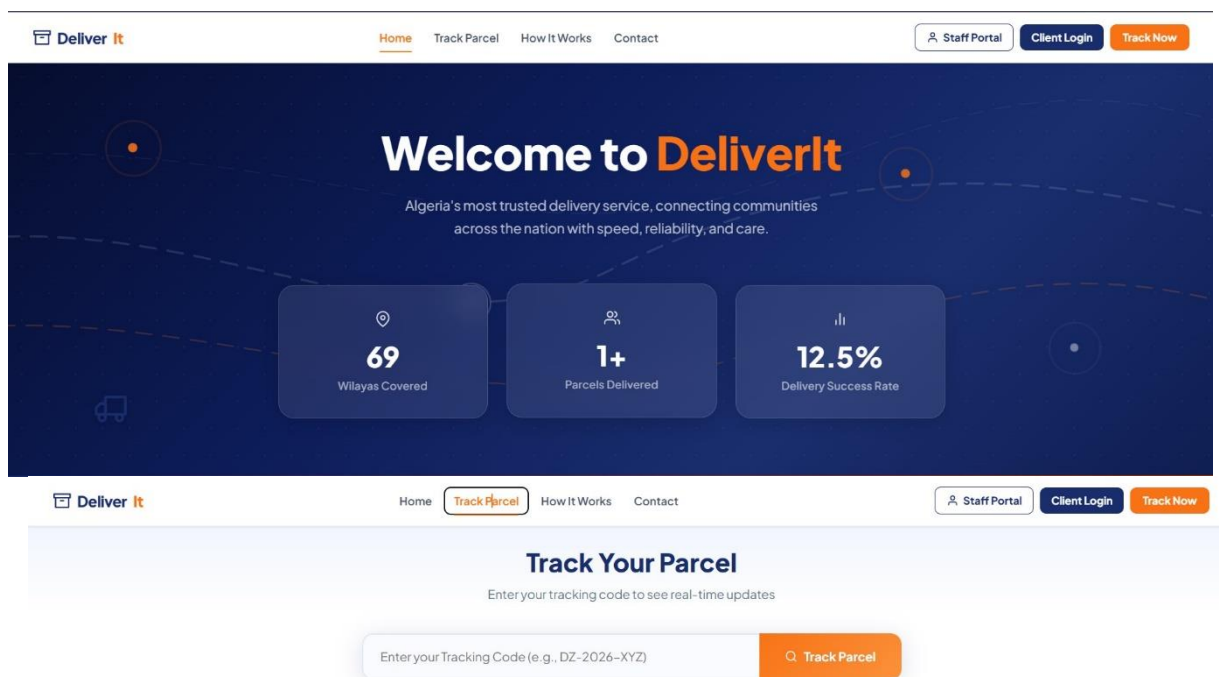


Figure 27 : Homepage Hero Section

The homepage features a hero section with a dark navy gradient background, real-time statistics (69 wilayas covered, number of delivered parcels, success rate), and an integrated parcel tracking form.

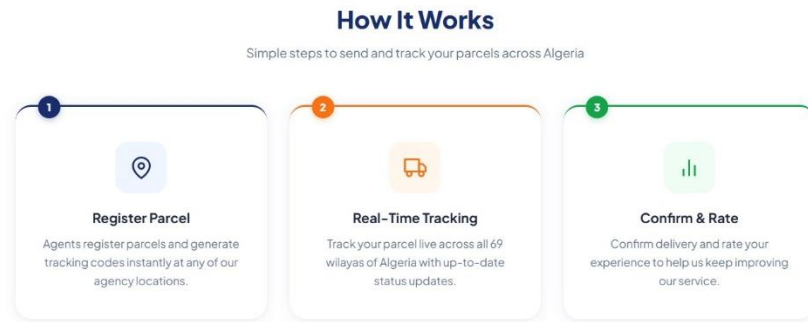


Figure 28 :How It Works Section

Three cards explain the delivery workflow to visitors: Register the Parcel, Real-Time Tracking, and Confirm & Rate.

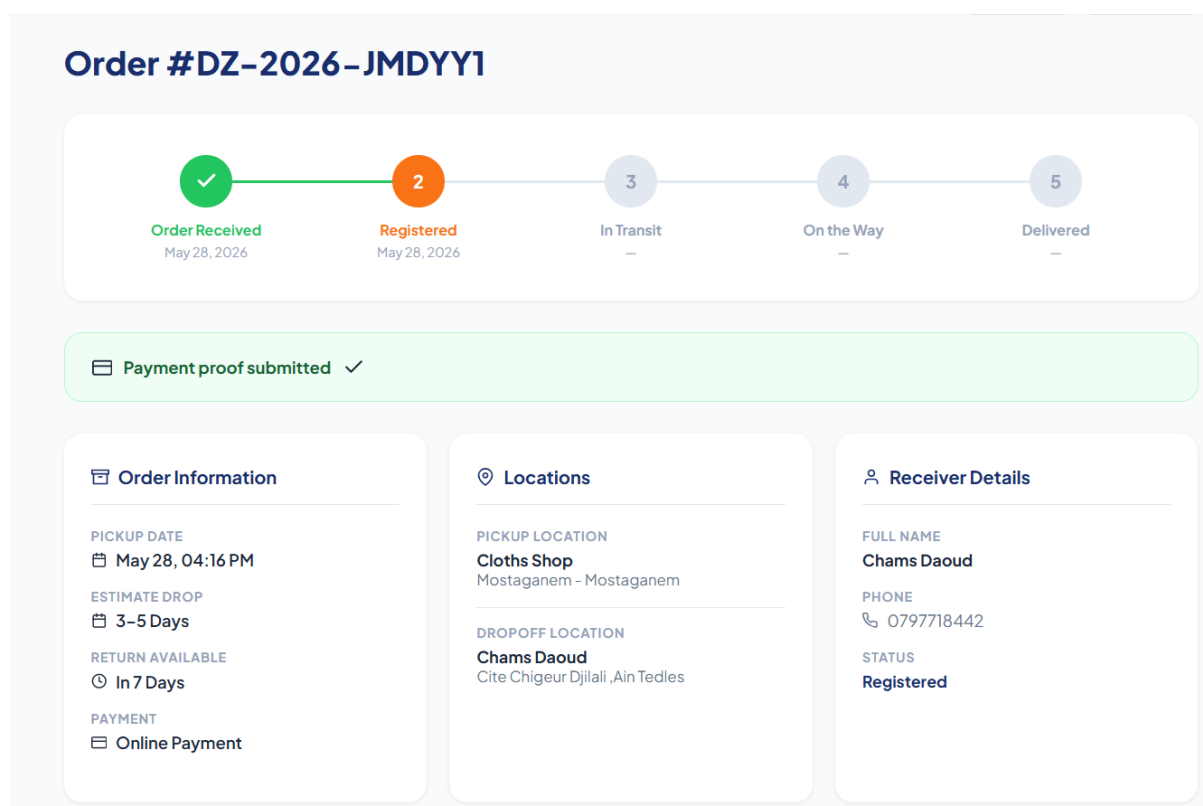


Figure 29 : Parcel Tracking Page

After entering a tracking code, the client is redirected to the tracking result page. This page displays the current parcel status through a progress stepper, sender and receiver information.

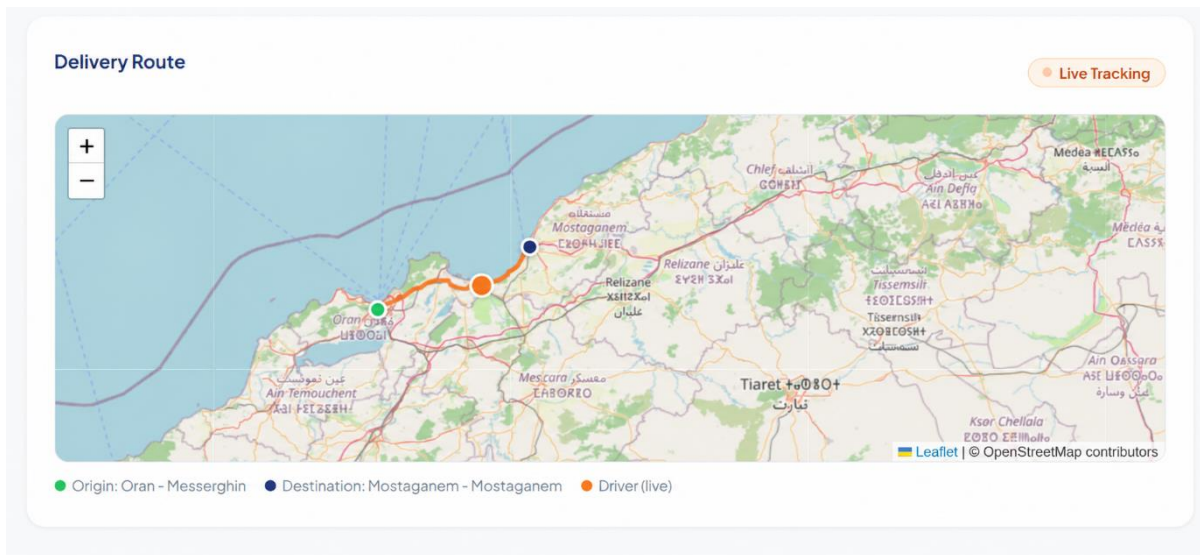


Figure 30 : Live GPS Tracking

When the parcel status is "Out for Delivery", an animated orange dot on the map indicates the driver's real-time position, updated every five seconds.

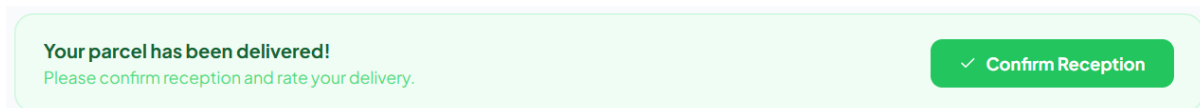


Figure 31 : Reception Confirmation and Rating

After delivery, the client can confirm reception and submit a rating from one to five stars along with an optional comment.

3.3.2 Authentication Interface

The figure displays three distinct user interface screens for client authentication, all featuring a dark blue header and footer with a central orange dot.

Welcome back
Track all your parcels in one place

Login Register

Email Address
your@email.com

Password
Your password

[Forgot password?](#)

Login

Create your account
Track all your parcels in one place

Login Register

Full Name
Your full name

Phone Number
+213 555 000 000

Email Address
your@email.com

Password
Min 8 characters

At least 8 characters

Create Account

Forgot Password?
Enter your email and we'll send you a reset link.

Email Address
your@email.com

Send Reset Link

Figure 32— Client Login Page

A dedicated login page for clients with an email and password form, a registration option, and a "Forgot password?" button that sends a reset link by email.

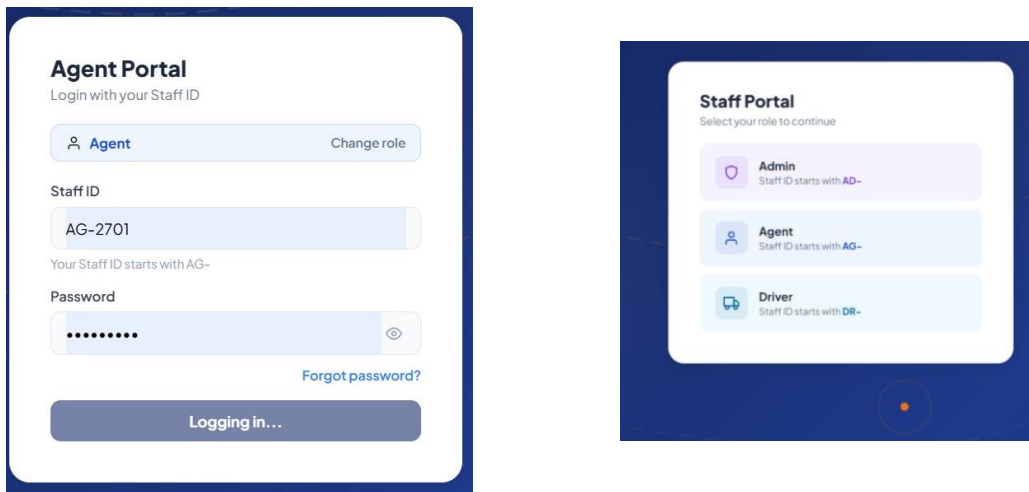


Figure 33 — Staff Login Page with role selection

A dedicated login page for staff members. The user first selects their role (Admin, Agent, or Driver), then enters their Staff ID and password. The system verifies that the Staff ID prefix matches the selected role (AD- for Admin, AG- for Agent, DR- for Driver).

3.3.3 Admin Dashboard

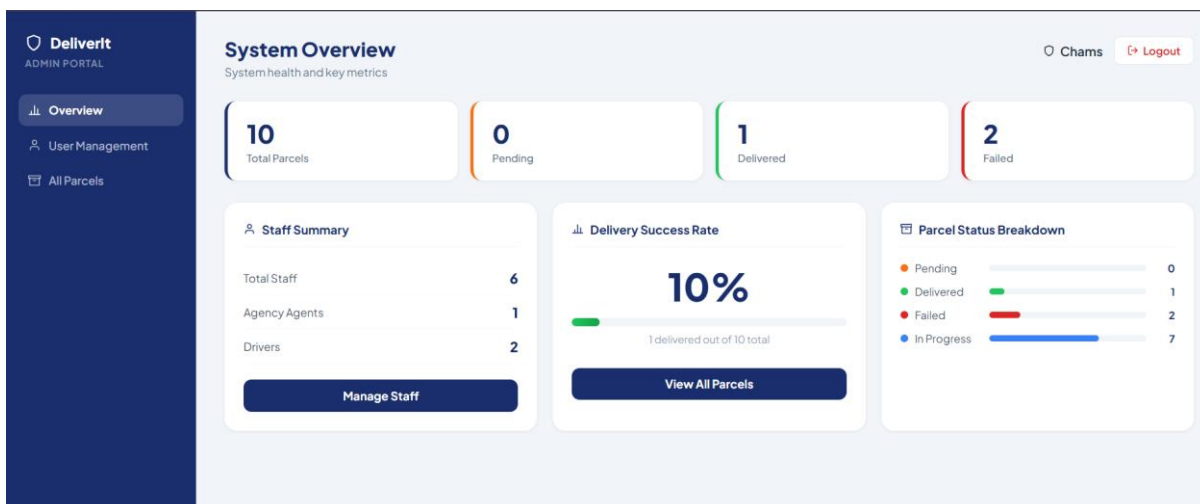


Figure 34 — Overview

The admin dashboard overview displays key system statistics: total parcels, parcels in transit, delivered, failed, and the overall delivery success rate with a progress bar.

Deliverit
ADMIN PORTAL

Overview

User Management

All Parcels

User Management
Manage agents, drivers and staff accounts

Chams Logout

All Roles

Create Staff Account

NAME	EMAIL	STAFF ID	PHONE	ROLE	ROUTE / WILAYA	JOINED	ACTIONS
Amine	daoudmohamedelamine@gmail.com	DR-271	0798625391	Driver	Mostaganem - Mostaganem	28/05/2026	Delete
Daoud Chams	daoudchams9@gmail.com	-	0798625391	Client	-	25/05/2026	Delete
Karim	thepolarios27@gmail.com	DR-3127	0556894231	Driver	Mostaganem ↔ Oran	25/05/2026	Delete
Sofa	saf12ben76@gmail.com	-	0797718442	Client	-	24/05/2026	Delete
Chams	daoudchams6@gmail.com	AG-2701	0556894591	Agent	-	23/05/2026	Delete
Chams	daoudchams39@gmail.com	AD-101	0797718442	Admin	-	23/05/2026	Delete

Figure 35 — Users Management

A table listing all agents, drivers, clients , and administrators with their Staff ID, role, and assigned route. The admin can create new staff accounts — an email with the login credentials is automatically sent to the new staff member.

Create Staff Account

Add a new agent or driver to the system

Role

Admin

Agent

Driver

Driver Type *

Intra-Wilaya
Same wilaya delivery

Inter-Wilaya
Between wilayas

Assigned Wilaya *

Select wilaya...

Assigned Commune *

Select wilaya first

Full Name *

Mohamed Amine

Staff ID *

AG-001

Phone *

0555 000 000

Email *

staff@deliverit.dz

Password *

Min 8 chars with letters and numbers

Cancel

Create Account

Figure 36 — Create Staff Account Modal

A modal form for creating a new staff account. For drivers, the admin selects the driver type (intra-wilaya or inter-wilaya) and assigns a specific wilaya or route (e.g., Mostaganem - Mazagran ↔ Oran Gdeyl).

3.3.4 Agent Dashboard

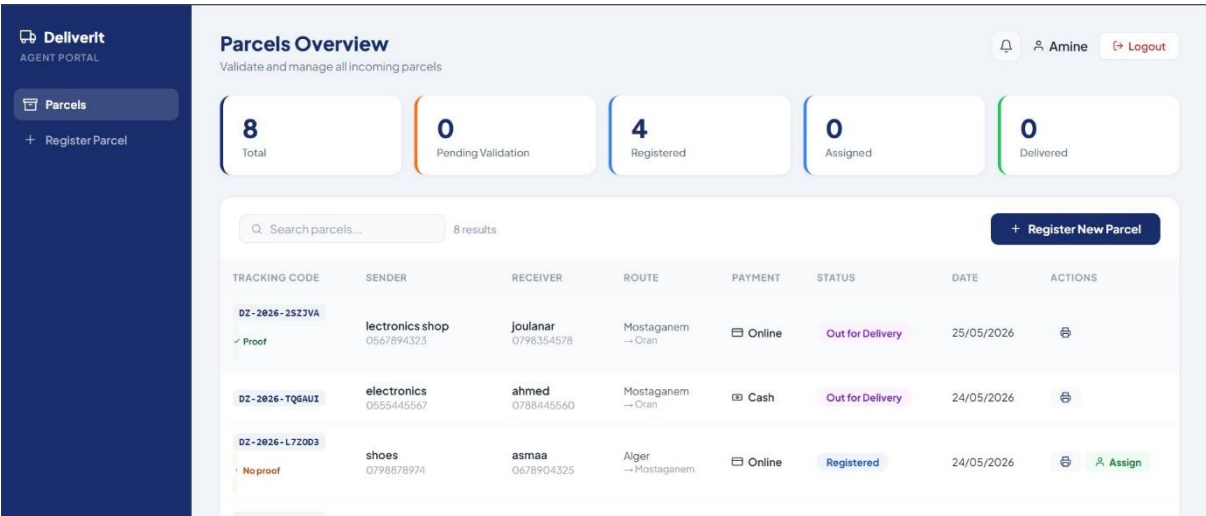


Figure 37 — Parcels Overview

The agent dashboard shows all registered parcels with their current status, payment method, and a payment proof indicator for online payments.

The screenshot shows the 'Register New Parcel' form. It has a header with a bell icon, the user name 'Chams', and a 'Logout' button. The form is divided into two main sections: 'Sender Information' and 'Parcel Details'. The 'Sender Information' section includes fields for 'Sender Full Name', 'Sender Phone', 'Origin Wilaya', and 'Origin Commune'. The 'Parcel Details' section includes a 'Description' text area, 'Weight (kg)', and 'Payment Method' (with radio buttons for 'Cash on Delivery' and 'Online Payment'). Below these are fields for 'Receiver Full Name', 'Receiver Phone', 'Destination Wilaya', 'Destination Commune', and 'Delivery Address'. At the bottom right are 'Cancel' and 'Register Parcel' buttons.

Figure 38 — Register Parcel Form

A complete parcel registration form including sender information (name, phone, wilaya, commune), receiver information (name, phone, wilaya, commune, delivery address), parcel description, weight, and payment method. The system automatically detects whether the delivery is intra-wilaya or inter-wilaya.

Parcel Validation

DZ-2026-JMDYY1

Review all information carefully before confirming

Same wilaya: Mostaganem

SENDER INFORMATION

FULL NAME

Cloths Shop

PHONE

0771432428

ORIGIN WILAYA

Mostaganem

RECEIVER INFORMATION

FULL NAME

Chams Daoud

PHONE

0797718442

DESTINATION WILAYA

Mostaganem

DELIVERY ADDRESS

Cite Chigreur
Djilali, Ain
Tedles

PARCEL DETAILS

DESCRIPTION

Not provided

WEIGHT

Not provided

PAYMENT

Online Payment

REGISTERED ON

28/05/2026, 16:16:38

Reject Parcel

Validate & Confirm

Parcel Validation

DZ-2026-JMDYY1

Review all information carefully before confirming

Tedles

PARCEL DETAILS

DESCRIPTION

Not provided

WEIGHT

Not provided

PAYMENT

Online Payment

REGISTERED ON

28/05/2026, 16:16:38

Payment Proof Submitted

Click to open full image

Reject Parcel

Validate & Confirm

Figure 40— Validation Modal

Before confirming a parcel, the agent reviews all information in a validation modal. For online payments, the payment proof uploaded by the client is displayed directly within the modal.

Assign Parcel to Driver

DZ-2026-A03UON

Inter-wilaya: Mostaganem → Oran

Showing inter-wilaya drivers only

○

Karim

0556894231 • DR-3127

Mostaganem ↔ Oran

Cancel

Assign Driver

Figure 41 — Driver Assignment Modal

The agent assigns the parcel to an available driver. The system automatically filters drivers based on their type and assigned route, showing only those compatible with the parcel's origin and destination.

Delivery Slip
Print or save as PDF to give to the client

Print / Save PDF
Close

Deliverit
Algeria's Trusted Delivery Network

Printed on
28 May 2026

TRACKING NUMBER
DZ-2026-AO3UON

Inter-Wilaya
Cash on Delivery

SENDER
Full Name
Computers Shop
Phone
0556893456
Origin Wilaya
Mostaganem

RECEIVER
Full Name
Yacine ben
Phone
0783673393
Destination Wilaya
Oran
Delivery Address
Es senia



PARCEL INFORMATION
Description
Weight
Registration Date
28/05/2026
<http://localhost:5173/?track=DZ-2026-AO3UON>

Scan to track

||| DZ-2026-AO3UON |||

Deliverit - info@deliverit.dz

Figure 42 — Delivery Slip with QR Code

The agent can print a delivery slip directly from the parcel list to hand to the sender physically.

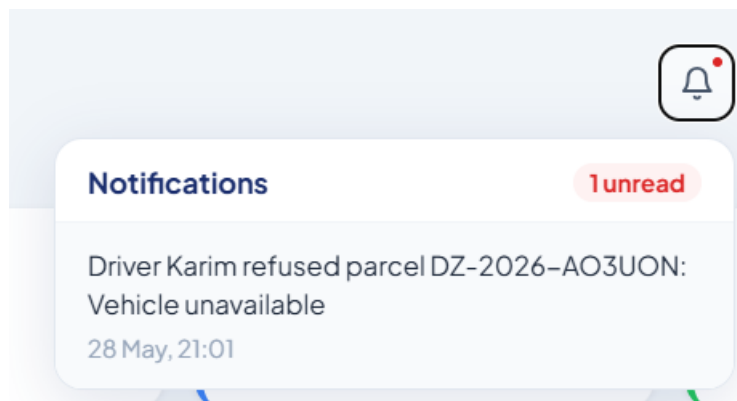


Figure 43 — Refusal Notifications

A bell icon with a badge in the agent's header displays notifications when a driver refuses an assigned parcel, including the driver's name, the refusal reason, and the tracking code.

3.3.5 Driver Dashboard

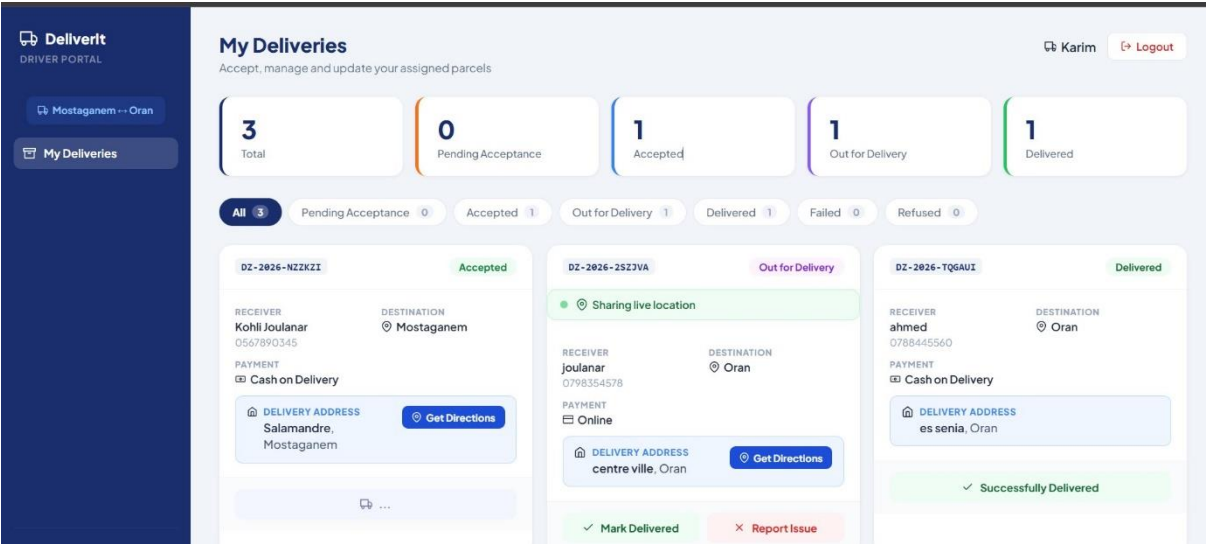


Figure 44 — Assigned Deliveries List

The driver dashboard displays all assigned parcels with Accept and Refuse buttons. The sidebar shows the driver's assigned route (e.g., Mostaganem ↔ Oran for inter-wilaya drivers, or a single wilaya for intra-wilaya drivers).

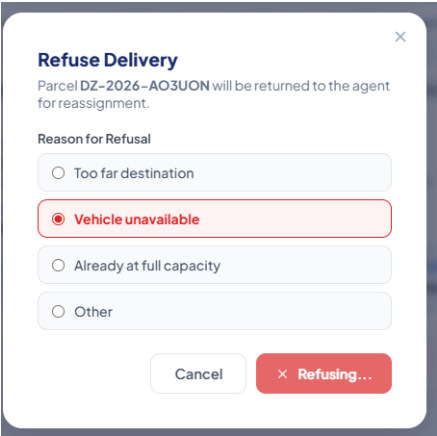
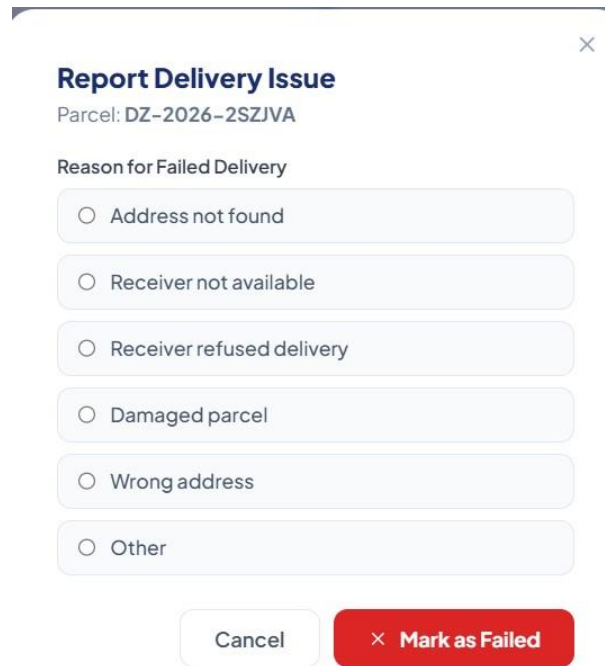


Figure 45 — Refusal Modal

When a driver refuses a parcel, a modal prompts them to select a reason: destination too far, vehicle unavailable, at full capacity, or other. The parcel status is then returned to "Registered" so the agent can reassign it.



Report Delivery Issue ×

Parcel: DZ-2026-2SZJVA

Reason for Failed Delivery

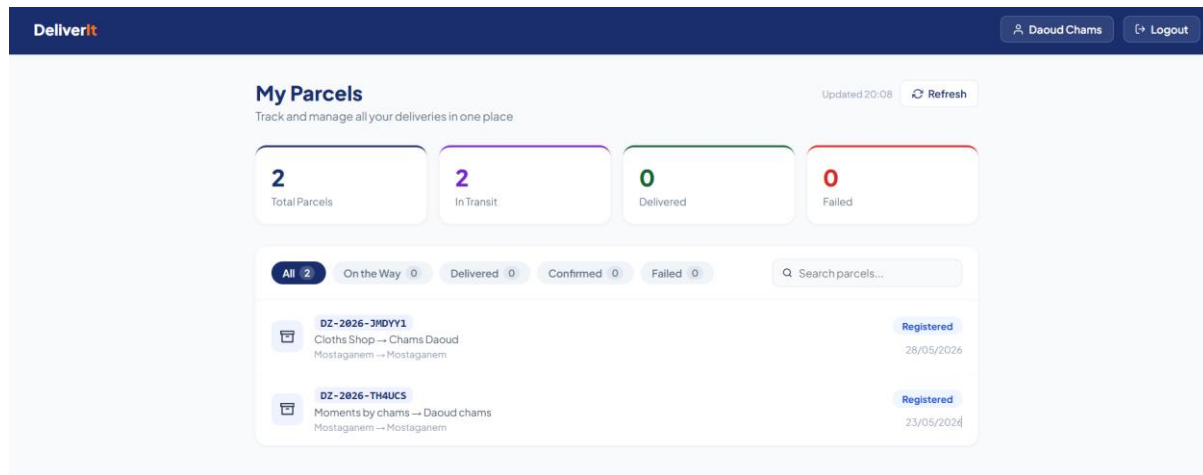
- ☐ Address not found
- ☐ Receiver not available
- ☐ Receiver refused delivery
- ☐ Damaged parcel
- ☐ Wrong address
- ☐ Other

Cancel × Mark as Failed

Figure 46 — Report Delivery Issue

If the delivery cannot be completed, the driver selects a failure reason (address not found, receiver not available, parcel refused, etc.). This reason is saved and displayed to the client on the tracking page.

3.3.6 Client Dashboard (After Login)



Deliverit Daoud Chams Logout

My Parcels Updated 20:08 Refresh

Track and manage all your deliveries in one place

Category	Count
Total Parcels	2
In Transit	2
Delivered	0
Failed	0

Filters: All (2) | On the Way (0) | Delivered (0) | Confirmed (0) | Failed (0) |

Parcel ID	Origin	Destination	Status	Date
DZ-2026-3HDYY1	Cloths Shop → Chams Daoud	Mostaganem → Mostaganem	Registered	28/05/2026
DZ-2026-TH4UCS	Moments by chams → Daoud chams	Mostaganem → Mostaganem	Registered	23/05/2026

Figure 47 — Client Dashboard

After logging in, the client sees an overview of all their parcels matched by phone number, with filter tabs by status (In Transit, Delivered, Confirmed, Failed) and a counter for each category.

Edit Profile
Update your personal information

✓ Profile updated successfully!

Full Name
Daoud Chams

Phone Number
0797718442

New Password (leave blank to keep current)
Min 8 chars with letters and numbers

Cancel ✓ Save Changes

Figure 48 — User Profile

The client can update their personal information including name, phone number, and password from the profile section.

3.4 Conclusion

This chapter presented the complete implementation of the **DeliverIt** delivery tracking system, from the choice of technologies to the final user interfaces. The application covers the full lifecycle of a parcel, from registration by the agent to reception confirmation by the client, including real-time GPS tracking, automated email notifications, and a structured role-based access system. The technologies used — React, Laravel, Supabase, and OpenStreetMap — form a modern, performant, and scalable architecture suited to the needs of the Algerian delivery market.

General Conclusion

This project allowed us to design and develop DeliverIt, a web application for delivery tracking and management in Algeria. Our goal was to solve a real problem — the lack of modern tools in the Algerian delivery sector — by building a complete system that covers the full lifecycle of a parcel, from registration to confirmation.

Through this work, we were able to apply what we learned during our studies in areas such as web development, database design, and software architecture. Working on a real-world problem helped us better understand how to translate user needs into technical solutions, and how to make decisions when facing practical challenges.

The result is a functional platform that serves four types of users — admin, agent, driver, and client with features including real-time GPS tracking, route-based driver assignment across all 69 wilayas, automated email notifications, and a public tracking page accessible to anyone.

This project is an important step in our academic journey. It can still be improved in the future by adding a mobile application for drivers, integrating electronic payment, deploying the system on a cloud server, and exploring the use of artificial intelligence for delivery time prediction.

We hope this work contributes in some way to the improvement of delivery services in Algeria.

Bibliography

- [1] Facebook Inc., *React — A JavaScript Library for Building User Interfaces*, Official Documentation, 2024. [Online]. Available: <https://react.dev>
- [2] Taylor Otwell, *Laravel — The PHP Framework for Web Artisans*, Official Documentation, v11, 2024. [Online]. Available: <https://laravel.com/docs>
- [3] Supabase Inc., *Supabase — The Open Source Firebase Alternative*, Official Documentation, 2024. [Online]. Available: <https://supabase.com/docs>
- [4] Vladimir Agafonkin, *Leaflet — an Open-Source JavaScript Library for Mobile-Friendly Interactive Maps*, Official Documentation, v1.9, 2024. [Online]. Available: <https://leafletjs.com>
- [5] OpenStreetMap Foundation, *OpenStreetMap — The Free Wiki World Map*, 2024. [Online]. Available: <https://www.openstreetmap.org>
- [6] Project OSRM, *OSRM — Open Source Routing Machine*, Official Documentation, 2024. [Online]. Available: <https://project-osrm.org>
- [7] Taylor Otwell, *Laravel Sanctum — Lightweight Authentication for SPAs and APIs*, Official Documentation, 2024. [Online]. Available: <https://laravel.com/docs/sanctum>
- [8] Evan You, *Vite — Next Generation Frontend Tooling*, Official Documentation, v5, 2024. [Online]. Available: <https://vitejs.dev>
- [9] Nominatim, *Nominatim — OpenStreetMap Geocoding API*, Official Documentation, 2024. [Online]. Available: <https://nominatim.openstreetmap.org>
- [10] Postman Inc., *Postman — API Platform for Building and Using APIs*, 2024. [Online]. Available: <https://www.postman.com>
- [14] Visual Studio Code, *VS Code — Code Editing Redefined*, Microsoft, 2024. [Online]. Available: <https://code.visualstudio.com>
- [15] npm Inc., *npm — Node Package Manager*, Official Documentation, 2024. [Online]. Available: <https://www.npmjs.com>